

Universidad Rafael Landívar.

Facultad de Ingeniería.

Ingeniería en Informática y Sistemas.

Virtualización - Sección 01.

Catedrático: Ing. Erick Oswaldo Marroquín Zamora



Proyecto #1

Documentación Técnica del inventario de Supermercado

Paula Ximena Navarro Ruano: 1006323

Guatemala, 30 de septiembre de 2026.

Introducción

Se implementó un inventario de productos con Vue, Flask y MongoDB, distribuido en cuatro contenedores administrados mediante Docker Compose. El gateway Nginx recibe las solicitudes, aplica HTTPS y dirige cada ruta hacia el servicio que corresponde. Los productos se guardan en un volumen nombrado para conservarlos al recrear los contenedores.

Alcance del informe

El informe describe la implementación y las pruebas del inventario. Incluye las verificaciones iniciales y las capturas de la configuración final, la construcción de imágenes, el CRUD desde Vue, la persistencia y el acceso HTTPS. Cada evidencia explica la actividad, el comando o configuración, su funcionamiento, el motivo de la prueba, el resultado y lo que demuestra.

Identificación de la solución

Carpeta y repositorio: proyecto_1_1006323_paula_ximena_navarro_ruano. Aplicación: Mi Súper. Rama de trabajo final: main. Último commit visible en la captura: 4216bb3.

Repositorio: https://github.com/IsPau04/proyecto_1_1006323_paula_ximena_navarro_ruano

Organización del informe

Las secciones corresponden a las etapas del enunciado: entorno, estructura, frontend, backend, MongoDB, variables, red, Compose, Nginx, certificados, imágenes, ejecución, CRUD, persistencia, HTTPS y logs. También se documentan la publicación en GitHub, los problemas resueltos y las conclusiones.

Arquitectura de la solución

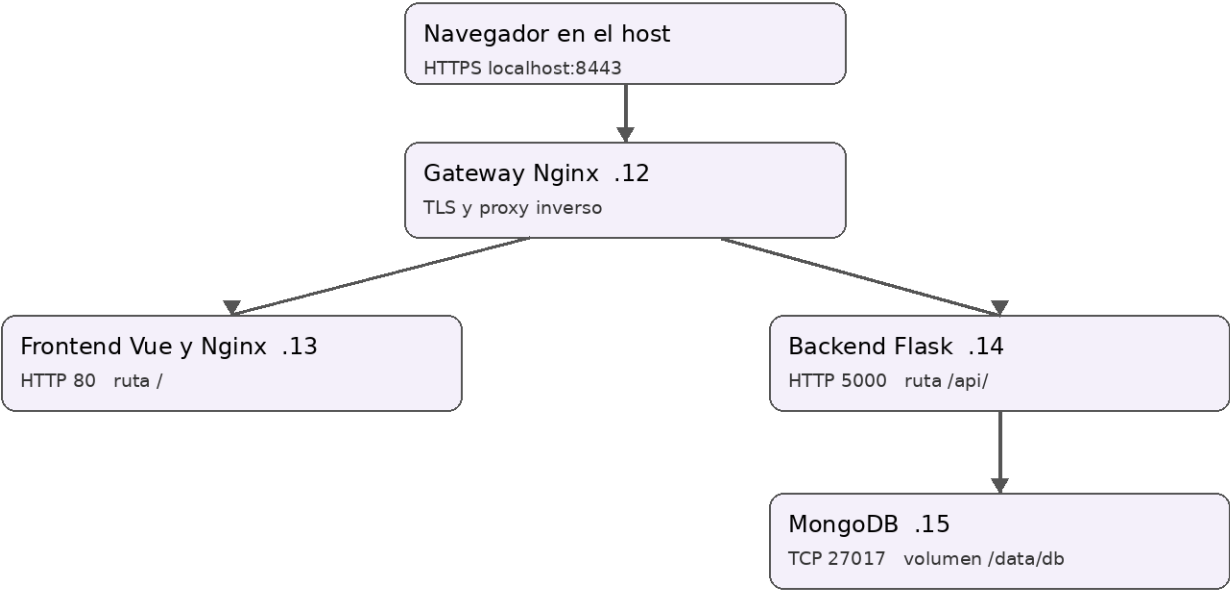


Figura 1. Diagrama elaborado a partir de la configuración final. Los últimos octetos pertenecen a 192.168.162.0/27.

Servicio	Responsabilidad	Acceso
gateway	HTTPS y enrutamiento	Host 8082 y 8443
frontend	Archivos compilados de Vue	Red Docker, puerto 80
backend	API REST y acceso a datos	Red Docker, puerto 5000
mongodb	Almacenamiento persistente	Red Docker, puerto 27017

Cuando se abre la página, el gateway envía la ruta / al frontend. Cuando Vue ejecuta fetch sobre /api/productos, la solicitud vuelve al mismo gateway y se dirige a Flask. Flask consulta o modifica MongoDB y devuelve JSON. Vue actualiza el formulario, la tabla y los indicadores con esa respuesta.

El cifrado TLS termina en Nginx. Los enlaces del gateway hacia frontend y backend usan HTTP dentro de la red Docker. La interfaz no se conecta directamente a MongoDB. Las direcciones 127.0.0.1 de los puertos publicados limitan el acceso al equipo local.

Etapa 1 Preparación del entorno

Trabajada en Windows con Docker Desktop para contenedores Linux, Git Bash y Visual Studio Code. Antes de desarrollar se comprueban las herramientas. La siguiente imagen corresponde a la verificación inicial recuperada; las versiones del host no se confunden con las versiones de las imágenes Docker.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Pau> docker --version
Docker version 29.6.2, build dfc4efb
PS C:\Users\Pau> docker compose version
Docker Compose version v5.3.1
PS C:\Users\Pau> git --version
git version 2.43.0.windows.1
PS C:\Users\Pau> node --version
v22.14.0
PS C:\Users\Pau> npm --version
10.9.2
PS C:\Users\Pau> python --version
Python 3.13.1
PS C:\Users\Pau> py -m pip --version
pip 26.0.1 from C:\Users\Pau\AppData\Roaming\Python\Python313\site-packa
ges\pip (python 3.13)
PS C:\Users\Pau> openssl version
OpenSSL 3.2.1 30 Jan 2024 (Library: OpenSSL 3.2.1 30 Jan 2024)
PS C:\Users\Pau> code --version
1.136.1
a44adf7f53e00964ab890f9f8758a334f1fc15bc
x64
PS C:\Users\Pau>
```

Figura 2. Verificación inicial de herramientas en PowerShell.

Comando	Versión registrada	Qué verifica
docker --version	29.6.2	Cliente Docker instalado
docker compose version	v5.3.1	Complemento de orquestación
git --version	2.43.0.windows.1	Control de versiones
openssl version	3.2.1	Generación y revisión de certificados
node --version	v22.14.0	Node instalado inicialmente
npm --version	10.9.2	Gestor de paquetes inicial
python --version	3.13.1	Python instalado en el host

Actividad y motivo: comprobar las herramientas necesarias antes de construir la infraestructura. Resultado: los comandos devolvieron versiones. La captura demuestra su disponibilidad inicial, aunque no acredita que Docker Desktop esté activo. Para ejecutar el proyecto final, Python y Node del host no son necesarios: se utilizan dentro de las imágenes.

Etapa 1 Verificación del motor Docker



```
Windows PowerShell
PS C:\Users\Pau> docker run --rm hello-world
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if the daemon is running: open npipe:////./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
PS C:\Users\Pau> docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\Users\Pau> |
```

Figura 3. Ejecución correcta de hello-world después de iniciar Docker Desktop.

Actividad: Comprobar que Docker puede crear y ejecutar un contenedor.

Comando o configuración: docker run --rm hello-world

Funcionamiento: Descarga la imagen si no existe, inicia el contenedor y lo elimina al terminar por --rm.

Motivo: La versión del cliente por sí sola no demuestra conexión con el motor.

Resultado: La segunda ejecución devolvió Hello from Docker. La primera correspondía al motor aún no disponible.

Qué demuestra: El cliente contactó al motor, se descargó la imagen y se ejecutó un contenedor correctamente.

Antes de levantar el inventario debo iniciar Docker Desktop y comprobar que usa contenedores Linux. Si la computadora se apaga, primero verifico el estado con docker compose ps y luego utilizo docker compose up -d -wait para recuperar los servicios.

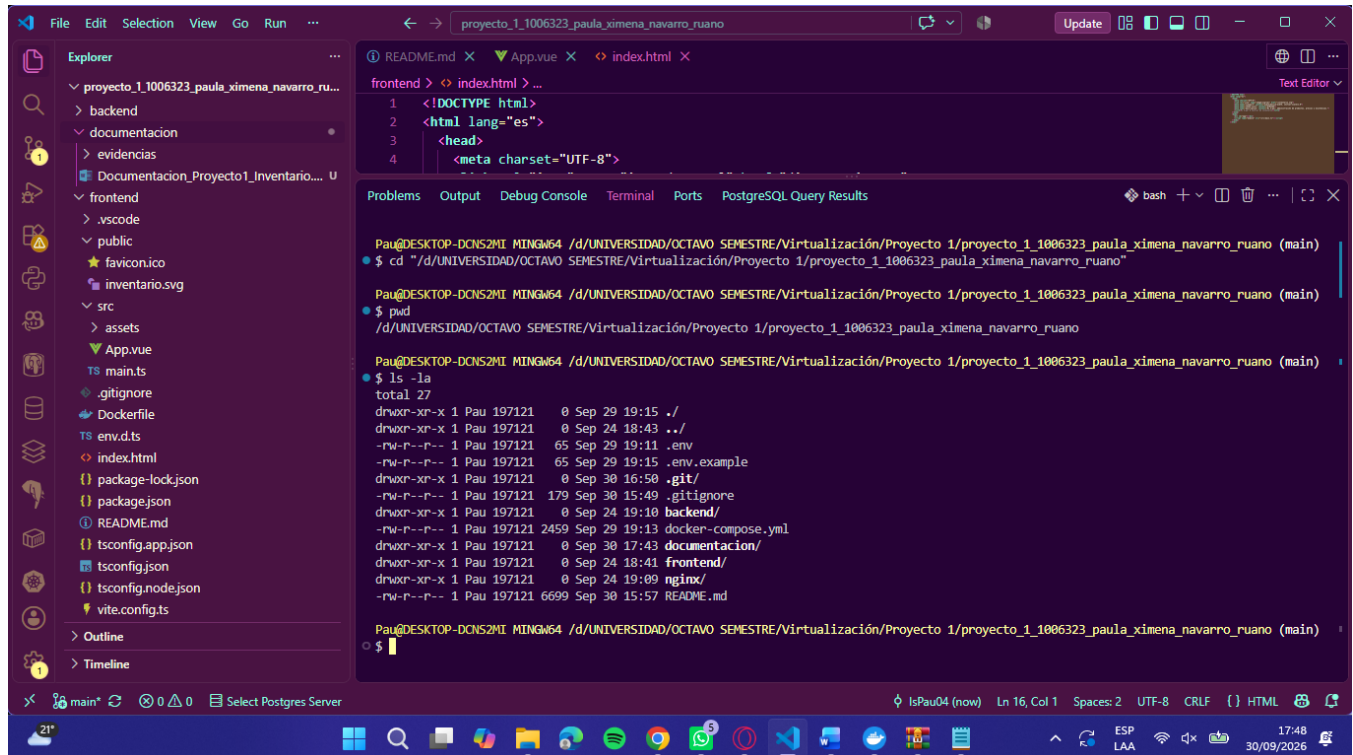
Etapa 2 Estructura del proyecto

La carpeta final sigue la nomenclatura solicitada y separa el código y la configuración según la responsabilidad de cada servicio. La evidencia de `pwd` y `ls -la` muestra la ubicación de trabajo y los archivos principales.

Ruta	Función
backend/app.py	Aplicación Flask, rutas CRUD y conexión con MongoDB
backend/requirements.txt	Flask 3.1.2 y PyMongo 4.15.1
backend/Dockerfile	Construcción de la imagen Python
frontend/src/App.vue	Interfaz, formulario, tabla, indicadores y solicitudes API
frontend/src/main.ts	Inicio de Vue y montaje en #app
frontend/index.html	Documento HTML base, título e idioma español
frontend/public/inventario.svg	Ícono local de la aplicación
frontend/package.json y package-lock.json	Scripts, dependencias y versiones resueltas
frontend/Dockerfile	Build con Node y servicio final con Nginx
nginx/nginx.conf.template	Plantilla de Nginx para el gateway HTTP y HTTPS, con variables de entorno.
nginx/certs/openssl.cnf	Identidad localhost y extensiones del certificado
nginx/certs/server.crt y server.key	Certificado público y llave privada local
docker-compose.yml	Definición de los cuatro servicios y sus recursos
.env y .env.example	Variables locales y ejemplo para reconstrucción
.gitignore	Exclusión de .env, llaves y archivos de desarrollo
README.md	Guía para instalación, operación y pruebas
documentacion/	Informe y capturas de evidencia
<pre>pwd ls -la ls backend frontend nginx nginx/certs</pre>	

`pwd` permite verificar que estoy en la carpeta correcta. `ls -la` lista archivos y directorios, incluidos `.env` y `.gitignore`. El último comando permite revisar las carpetas principales. La captura siguiente muestra la ubicación final y su estructura.

Ubicación y estructura del proyecto



Evidencia 1. Ubicación y estructura del proyecto.

Actividad: Revisar la carpeta de trabajo.

Comando o configuración: `pwd` y `ls -la`

Funcionamiento: Imprime la ruta actual y lista archivos y directorios.

Motivo: Comprobar que la ejecución corresponde a la carpeta final.

Resultado: La ruta muestra el proyecto individual y aparecen backend, frontend, nginx, documentacion y los archivos de configuración.

Qué demuestra: La ubicación y la organización de la solución.

Etapa 3 Frontend en Vue

El frontend está desarrollado con Vue 3, TypeScript, HTML y CSS. App.vue concentra la interfaz: encabezado, tarjetas de resumen, formulario, búsqueda, tabla de productos, avisos y diálogo de confirmación. El diseño usa colores morados y adapta la distribución a pantallas pequeñas.

Creación y ejecución de la aplicación

El proyecto utiliza la estructura del generador de Vue y Vite. Para generar una base equivalente se puede ejecutar `npm create vue@latest frontend` y seleccionar TypeScript. No se conserva la captura del asistente original; este comando se incluye como referencia de creación, no como evidencia de su ejecución. Para reconstruir esta aplicación debo clonar el código final, ya que generar una base nueva no reproduce sus componentes.

main.ts crea la aplicación e importa los estilos generales. index.html incluye el elemento `#app` y carga el punto de entrada. App.vue usa `ref` para el estado, `computed` para la búsqueda y los indicadores, y `onMounted` para consultar los productos al cargar la vista.

Formulario y validaciones

Campo o acción	Regla implementada
Nombre	Obligatorio; se eliminan espacios de los extremos
Precio	Número finito, no negativo y con máximo dos decimales
Stock	Entero seguro mayor o igual a cero
Guardar	Evita envíos duplicados mientras hay una operación pendiente
Eliminar	Solicita confirmación en un diálogo
Consulta	Muestra carga, ausencia de registros y errores de respuesta

La búsqueda compara nombres sin distinguir mayúsculas ni acentos. Los indicadores se calculan con la lista consultada: cantidad de productos, suma del stock y valor total de precio por stock. Estos cálculos corresponden al inventario actual; no son utilidades ni ventas.

Comunicación con el backend

Vue utiliza rutas relativas del mismo origen: `GET /api/productos` para listar, `POST` para crear, `PUT /api/productos/<id>` para editar y `DELETE` para eliminar. Los envíos de datos usan JSON y `Content-Type: application/json`. Nginx recibe esas rutas y las dirige a Flask, por lo que el navegador no necesita conocer la IP del backend.

El cliente controla el estado HTTP y los datos recibidos, muestra mensajes comprensibles y aplica un tiempo máximo de espera. Las validaciones de la interfaz ayudan al usuario; el backend actual tiene validación básica de campos y no reproduce todas las restricciones numéricas del frontend.

Etapa 3 Evidencia del frontend final

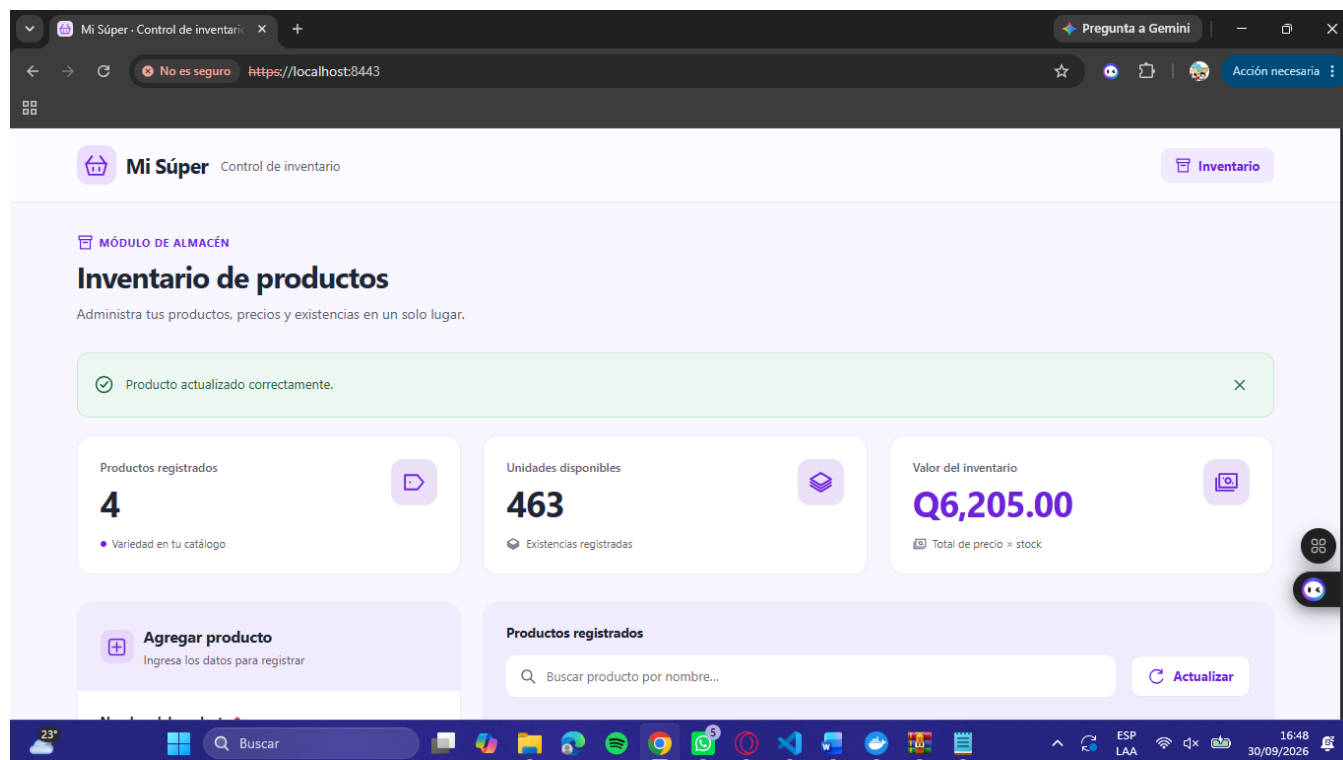


Figura 4. Interfaz final y mensaje de actualización correcta mediante HTTPS.

Actividad: Consultar el inventario y actualizar un producto desde Vue.

Comando o configuración: <https://localhost:8443> y formulario de edición en App.vue.

Funcionamiento: Vue envía la modificación por PUT, vuelve a consultar los datos y actualiza la vista.

Motivo: Verificar la operación desde la interfaz requerida por el enunciado.

Resultado: La pantalla muestra Producto actualizado correctamente, 4 productos, 463 unidades y Q6,205.00.

Qué demuestra: La aplicación está accesible por HTTPS y muestra una actualización exitosa. La captura no permite conocer por sí sola el valor anterior del producto.

Etapa 3 Formulario y productos registrados

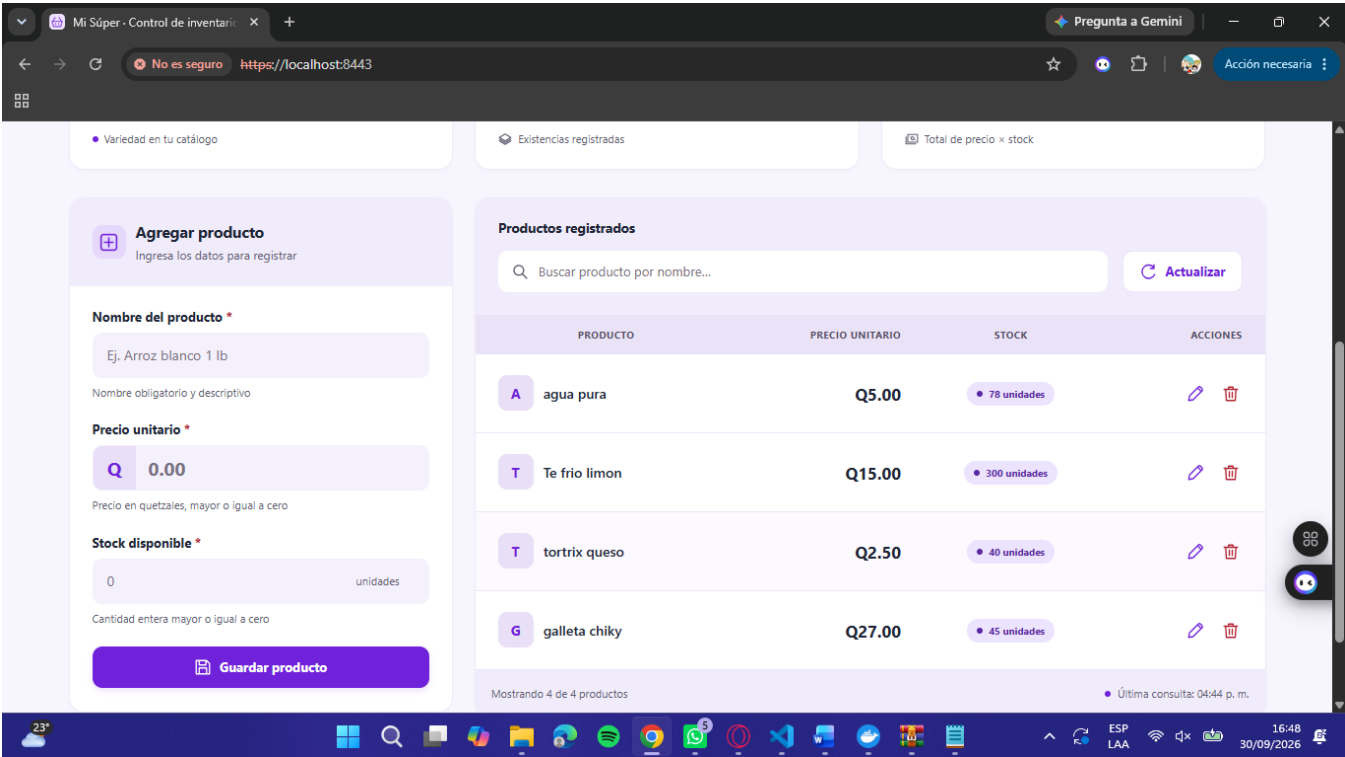


Figura 5. Formulario y tabla con los registros recuperados.

La tabla muestra agua pura, Te frio limon, tortrix queso y galleta chiky. Los botones de editar y eliminar están disponibles. Su presencia no sustituye una captura del resultado de crear o eliminar; esas evidencias se detallan en la etapa 13.

Etapas 3 y 11 Dockerfile del frontend

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

FROM node:22-alpine AS build inicia la etapa de construcción y le asigna un nombre. WORKDIR /app establece el directorio de trabajo. COPY package*.json ./ copia las definiciones de dependencias antes que el resto del código, para aprovechar la caché. RUN npm install instala los paquetes.

COPY . . incorpora la aplicación. RUN npm run build ejecuta el script definido en package.json, que realiza comprobación de tipos y compilación con Vite. El resultado se guarda en dist. Si esta instrucción falla, la construcción se detiene y no se genera una nueva imagen final.

El segundo FROM nginx:alpine inicia una imagen independiente para ejecución. COPY --from=build copia solamente dist al directorio /usr/share/nginx/html. Así, el contenedor final sirve archivos compilados y no necesita un servidor de desarrollo ni Node en ejecución.

EXPOSE 80 declara el puerto usado por ese Nginx, pero no lo publica en Windows. CMD ejecuta Nginx en primer plano con daemon off, de modo que su proceso mantiene activo el contenedor. La configuración predeterminada del Nginx de esta imagen sirve la aplicación estática; el proxy y TLS están en el gateway separado.

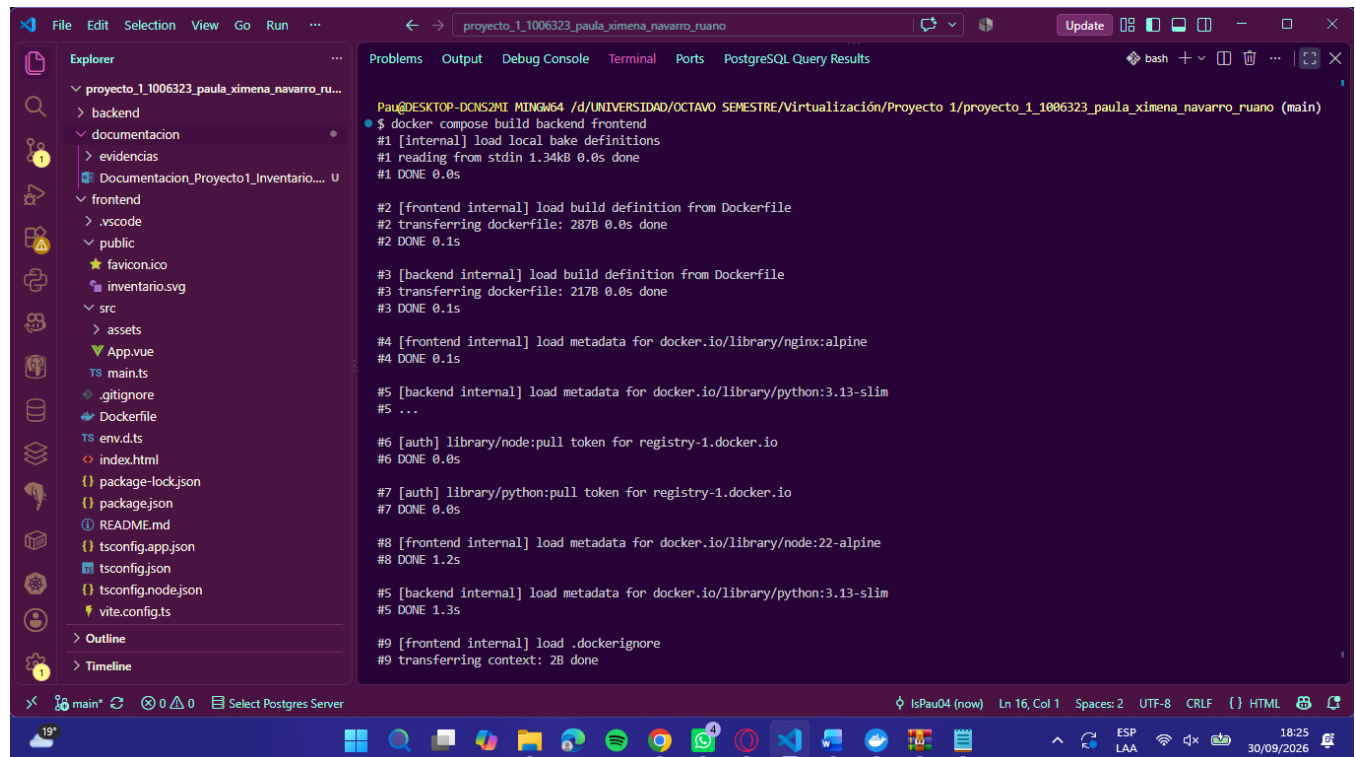
Construcción de producción

```
docker compose build frontend
docker images
```

El primer comando construye la imagen del servicio frontend usando su Dockerfile. El segundo lista las imágenes locales. Compose la etiqueta como proyecto1-frontend:1.0. La compilación final también se ejecuta al usar docker compose up -d --build.

Las capturas de construcción identifican los servicios backend y frontend y muestran ambas imágenes con el estado Built. docker images confirma las etiquetas proyecto1-backend:1.0 y proyecto1-frontend:1.0. Las etapas CACHED reutilizan resultados previos y no constituyen un error.

Inicio de la construcción de imágenes



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose build backend frontend
#1 [internal] load local bake definitions
#1 reading from stdin 1.34kB 0.0s done
#1 DONE 0.0s

#2 [frontend internal] load build definition from Dockerfile
#2 transferring dockerfile: 287B 0.0s done
#2 DONE 0.1s

#3 [backend internal] load build definition from Dockerfile
#3 transferring dockerfile: 217B 0.0s done
#3 DONE 0.1s

#4 [frontend internal] load metadata for docker.io/library/nginx:alpine
#4 DONE 0.1s

#5 [backend internal] load metadata for docker.io/library/python:3.13-slim
#5 ...

#6 [auth] library/node:pull token for registry-1.docker.io
#6 DONE 0.0s

#7 [auth] library/python:pull token for registry-1.docker.io
#7 DONE 0.0s

#8 [frontend internal] load metadata for docker.io/library/node:22-alpine
#8 DONE 1.2s

#5 [backend internal] load metadata for docker.io/library/python:3.13-slim
#5 DONE 1.3s

#9 [frontend internal] load .dockerignore
#9 transferring context: 2B done
```

Evidencia 2. Inicio de la construcción de imágenes.

Actividad: Construir ambos servicios desde sus Dockerfiles.

Comando o configuración: docker compose build backend frontend

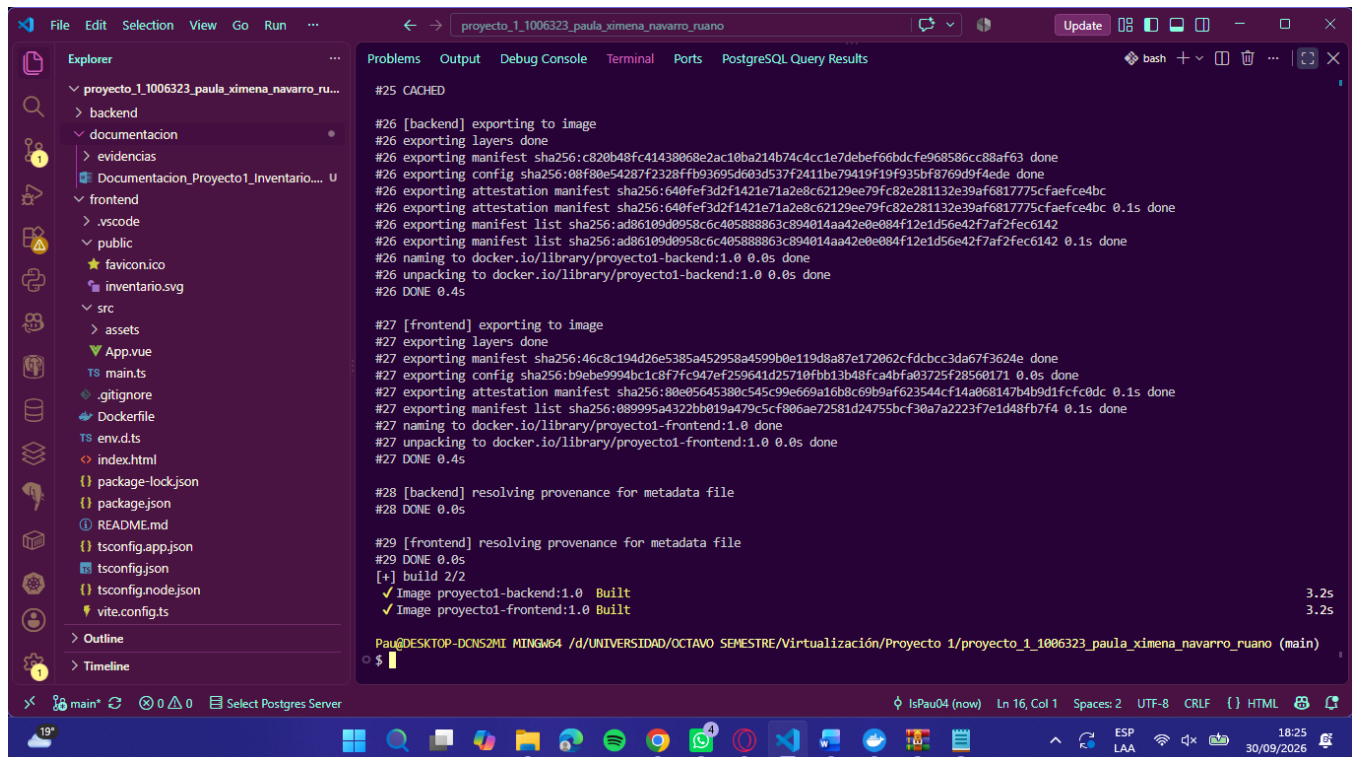
Funcionamiento: Docker carga los Dockerfiles y resuelve las imágenes base de Python, Node y Nginx.

Motivo: Generar las imágenes de la aplicación.

Resultado: La captura muestra el comando y las etapas iniciales de construcción.

Qué demuestra: La construcción usa los servicios backend y frontend de Compose.

Finalización de la construcción de imágenes



```
#25 CACHED

#26 [backend] exporting to image
#26 exporting layers done
#26 exporting manifest sha256:c820b48fc41438068e2ac10ba214b74c4cc1e7debf66bdcfe968586cc88af63 done
#26 exporting config sha256:08f00e54287f2328fb93695d603d537f2411be79419f19f935bf8769d9f4ede done
#26 exporting attestation manifest sha256:640fef3d2f1421e71a2e8c62129ee79fc82e281132e39af6817775cfaefce4bc done
#26 exporting attestation manifest sha256:640fef3d2f1421e71a2e8c62129ee79fc82e281132e39af6817775cfaefce4bc 0.1s done
#26 exporting manifest list sha256:ad86109d0958c6c405888863c894014aa42e0e084f12e1d56e42f7af2fec6142 done
#26 exporting manifest list sha256:ad86109d0958c6c405888863c894014aa42e0e084f12e1d56e42f7af2fec6142 0.1s done
#26 naming to docker.io/library/proyecto1-backend:1.0 0.0s done
#26 unpacking to docker.io/library/proyecto1-backend:1.0 0.0s done
#26 DONE 0.4s

#27 [frontend] exporting to image
#27 exporting layers done
#27 exporting manifest sha256:46c8c194d26e5385a452958a4599b0e119d8a87e172062cfdcbcc3da67f3624e done
#27 exporting config sha256:b9ebe9994bc1c8f7fc947ef259641d25710fb13b48fca4bfa03725f28560171 0.0s done
#27 exporting attestation manifest sha256:80e05645380c545c99e669a16b8c69b9af623544cf14a068147b4b9d1fcfc0dc 0.1s done
#27 exporting manifest list sha256:089995a4322bb019a479c5cf806ae72581d24755bcf30a7a2223f7e1d48fb7f4 0.1s done
#27 naming to docker.io/library/proyecto1-frontend:1.0 done
#27 unpacking to docker.io/library/proyecto1-frontend:1.0 0.0s done
#27 DONE 0.4s

#28 [backend] resolving provenance for metadata file
#28 DONE 0.0s

#29 [frontend] resolving provenance for metadata file
#29 DONE 0.0s
[+] build 2/2
✓ Image proyecto1-backend:1.0 Built 3.2s
✓ Image proyecto1-frontend:1.0 Built 3.2s

Pau@DESKTOP-DCNS2MI MINGW64 /d:/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$
```

Evidencia 3. Finalización de la construcción de imágenes.

Actividad: Verificar el resultado del build.

Comando o configuración: docker compose build backend frontend

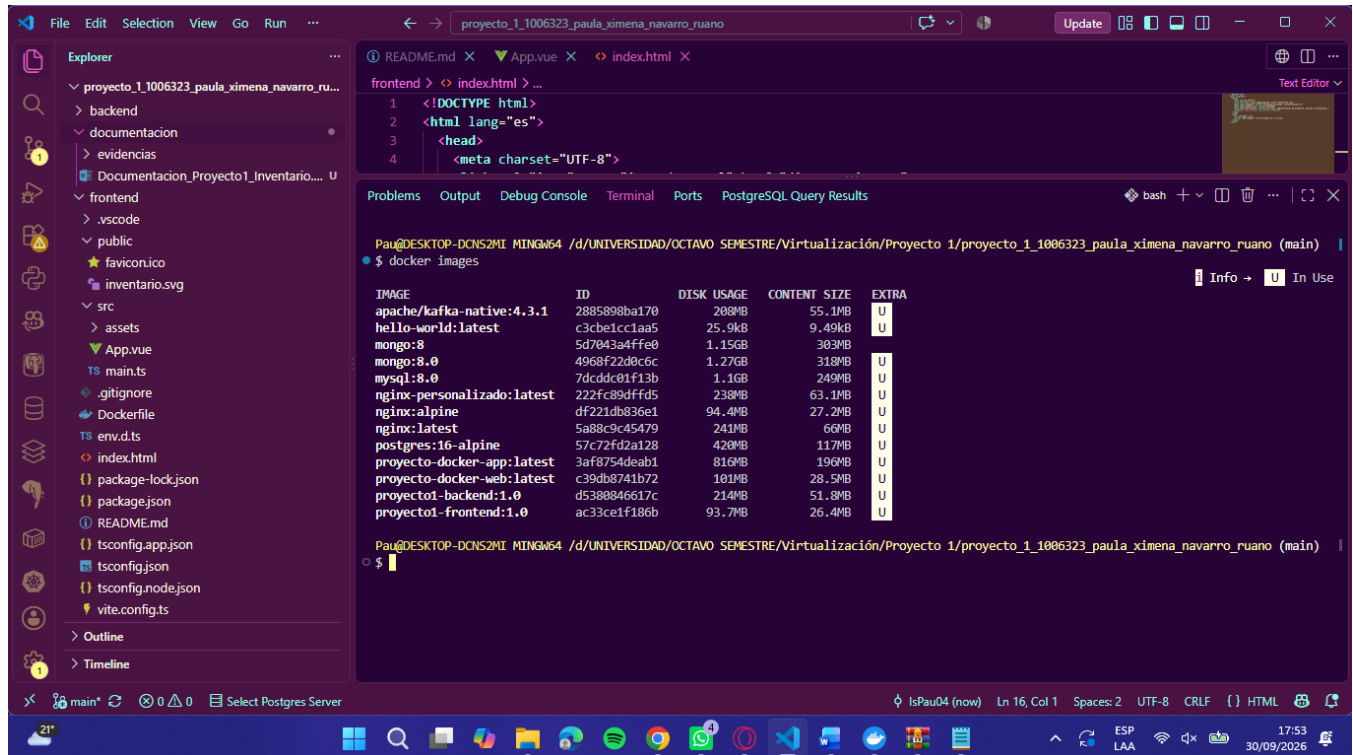
Funcionamiento: Exporta y etiqueta las imágenes construidas; utiliza caché cuando corresponde.

Motivo: Confirmar que la construcción termina correctamente.

Resultado: proyecto1-backend:1.0 y proyecto1-frontend:1.0 aparecen Built.

Qué demuestra: Los dos Dockerfiles producen imágenes utilizables.

Imágenes Docker del proyecto



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with folders like 'backend', 'documentation', 'evidencias', 'frontend', and 'public'. The terminal displays the output of the 'docker images' command, listing various Docker images and their details.

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
apache/kafka-native:4.3.1	2885898ba170	208MB	55.1MB	U
hello-world:latest	c3cbe1cc1aa5	25.9kB	9.49kB	U
mongo:8	5d7043a4ffe0	1.15GB	303MB	U
mongo:8.0	4968f22d0c6c	1.27GB	318MB	U
mysql:8.0	7dcddc01f13b	1.1GB	249MB	U
nginx-personalizado:latest	222fc89dff5	238MB	63.1MB	U
nginx:alpine	df221db836e1	94.4MB	27.2MB	U
nginx:latest	5a88c9c45479	241MB	66MB	U
postgres:16-alpine	57c72fd2a128	420MB	117MB	U
projecto-docker-app:latest	3af8754deab1	816MB	196MB	U
projecto-docker-web:latest	c39db8741b72	101MB	28.5MB	U
projecto1-backend:1.0	d5380846617c	214MB	51.8MB	U
projecto1-frontend:1.0	ac33ce1f186b	93.7MB	26.4MB	U

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$
```

Evidencia 4. Imágenes Docker del proyecto.

Actividad: Listar las imágenes locales.

Comando o configuración: docker images

Funcionamiento: Muestra nombres, etiquetas, identificadores y tamaños.

Motivo: Identificar las imágenes propias entre las imágenes del equipo.

Resultado: Aparecen proyecto1-backend:1.0 y proyecto1-frontend:1.0.

Qué demuestra: Las imágenes del backend y frontend están disponibles localmente.

Etapa 4 Backend Flask y API REST

El backend usa Flask para recibir solicitudes HTTP y PyMongo para trabajar con MongoDB. requirements.txt fija Flask==3.1.2 y pymongo==4.15.1. La instalación inicial de esas versiones quedó registrada en la salida de pip conservada; el Dockerfile realiza la misma instalación dentro de la imagen.

Método y ruta	Entrada	Respuesta normal
GET /api/saludo	Sin cuerpo	200 y mensaje de disponibilidad
GET /api/productos	Sin cuerpo	200 y lista de productos
GET /api/productos/<id>	ID en URL	200 y producto
POST /api/productos	nombre, precio y stock en JSON	201 y producto creado
PUT /api/productos/<id>	ID y campos válidos en JSON	200 y producto actualizado
DELETE /api/productos/<id>	ID en URL	200 y mensaje de eliminación

Ejemplo del contrato JSON

```
{"nombre": "Arroz blanco", "precio": 12.50, "stock": 20}
```

Es un ejemplo de solicitud, no un registro observado. Las respuestas de productos incluyen id, nombre, precio y stock. La función producto_json convierte el ObjectId de MongoDB en una cadena para enviarlo como JSON.

Manejo de errores

POST devuelve 400 cuando faltan datos o los campos obligatorios. PUT devuelve 400 si no recibe datos o no hay campos válidos. GET individual, PUT y DELETE devuelven 404 cuando el ID válido no corresponde a un producto. Las rutas que convierten ObjectId capturan excepciones y devuelven 400 con un mensaje de ID inválido.

Esa captura de excepciones es amplia y puede clasificar otros fallos como un ID inválido. GET de la lista y POST no tienen un tratamiento específico de indisponibilidad de MongoDB. La configuración actual inicia el servidor integrado de Flask con FLASK_DEBUG=false; este alcance es académico y local. Para un despliegue público sería necesario usar un servidor WSGI y fortalecer validaciones y control de acceso.

Conexión

os.getenv obtiene MONGO_HOST, MONGO_PORT y MONGO_DATABASE. El puerto se convierte a entero. MongoClient recibe el host mongodb y el puerto 27017; luego se selecciona proyecto1_db y la colección productos. El backend escucha en 0.0.0.0:5000 dentro de su contenedor, permitiendo que el gateway lo contacte.

Etaapa 4 Dockerfile y prueba inicial del backend

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

FROM python:3.13-slim aporta Python con una base reducida. WORKDIR /app establece el directorio. COPY requirements.txt . y RUN pip install --no-cache-dir -r requirements.txt instalan las dependencias sin conservar la caché de pip. COPY app.py . agrega el código. EXPOSE 5000 describe el puerto interno y CMD ["python", "app.py"] inicia Flask.

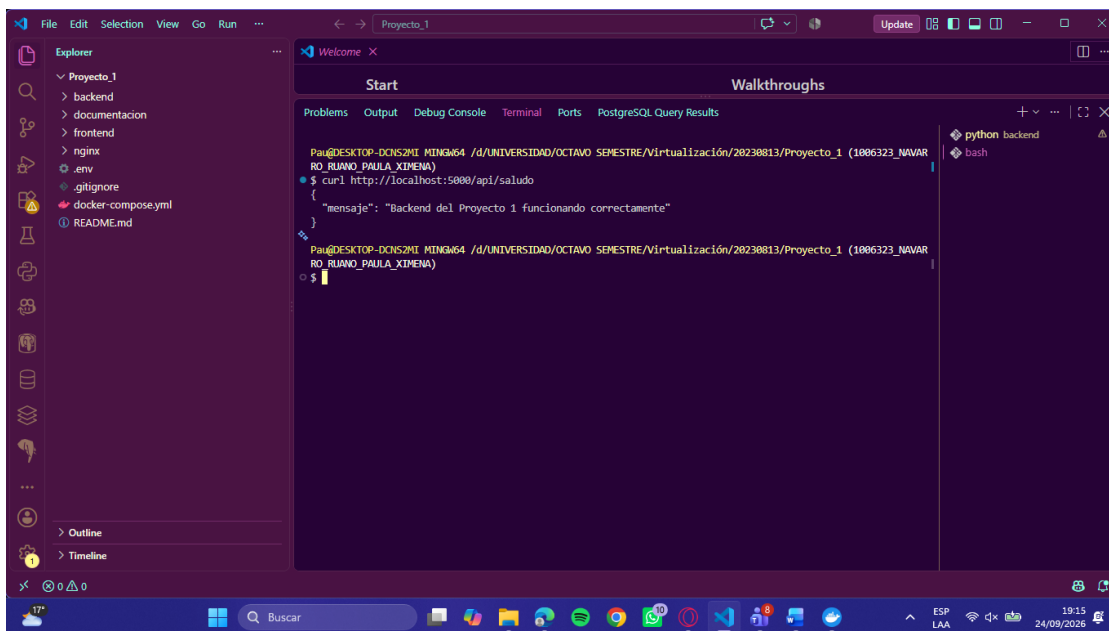


Figura 6. Prueba inicial de la ruta de saludo durante el desarrollo.

Actividad: Verificar la respuesta de Flask durante la preparación inicial.

Comando o configuración: curl http://localhost:5000/api/saludo

Funcionamiento: Realiza una solicitud GET a la ruta de diagnóstico.

Motivo: Separar la prueba inicial del backend de la integración posterior.

Resultado: Se recibió el mensaje Backend del Proyecto 1 funcionando correctamente.

Qué demuestra: El código Flask respondió durante el desarrollo. Esta prueba fue en el host; el despliegue final utiliza el backend dentro de Docker y acceso mediante el gateway.

Etapa 5 MongoDB y almacenamiento

MongoDB usa la imagen mongo:8.0 y el contenedor mongod. La base seleccionada por Flask es proyecto1_db y la colección es productos. No se configura autenticación en esta solución local. El servicio carece de ports, por lo que el puerto 27017 no se publica al host.

Campo almacenado	Tipo utilizado	Función
_id	ObjectId	Identificador generado por MongoDB
nombre	Texto	Nombre del producto
precio	Número	Precio unitario en quetzales
stock	Número entero desde Vue	Cantidad disponible

El documento se inserta mediante insert_one. La lectura usa find o find_one, la actualización utiliza update_one con \$set y la eliminación usa delete_one. La API transforma _id en id al responder. MongoDB crea efectivamente la base y la colección cuando se almacenan datos; MONGO_INITDB_DATABASE por sí sola no incorpora productos.

Volumen nombrado

El recurso mongo_data de Compose apunta al volumen externo proyecto1_mongo_data y se monta en /data/db. En ese directorio MongoDB guarda sus archivos de datos. El volumen tiene un ciclo de vida independiente de los contenedores; conservarlo permite recrear el servicio sin perder los productos.

```
docker volume ls
docker volume inspect proyecto1_mongo_data
docker inspect mongodb --format '{{range .Mounts}}{{.Type}} {{.Name}} ->
{{.Destination}}{{println}}{{end}}'
```

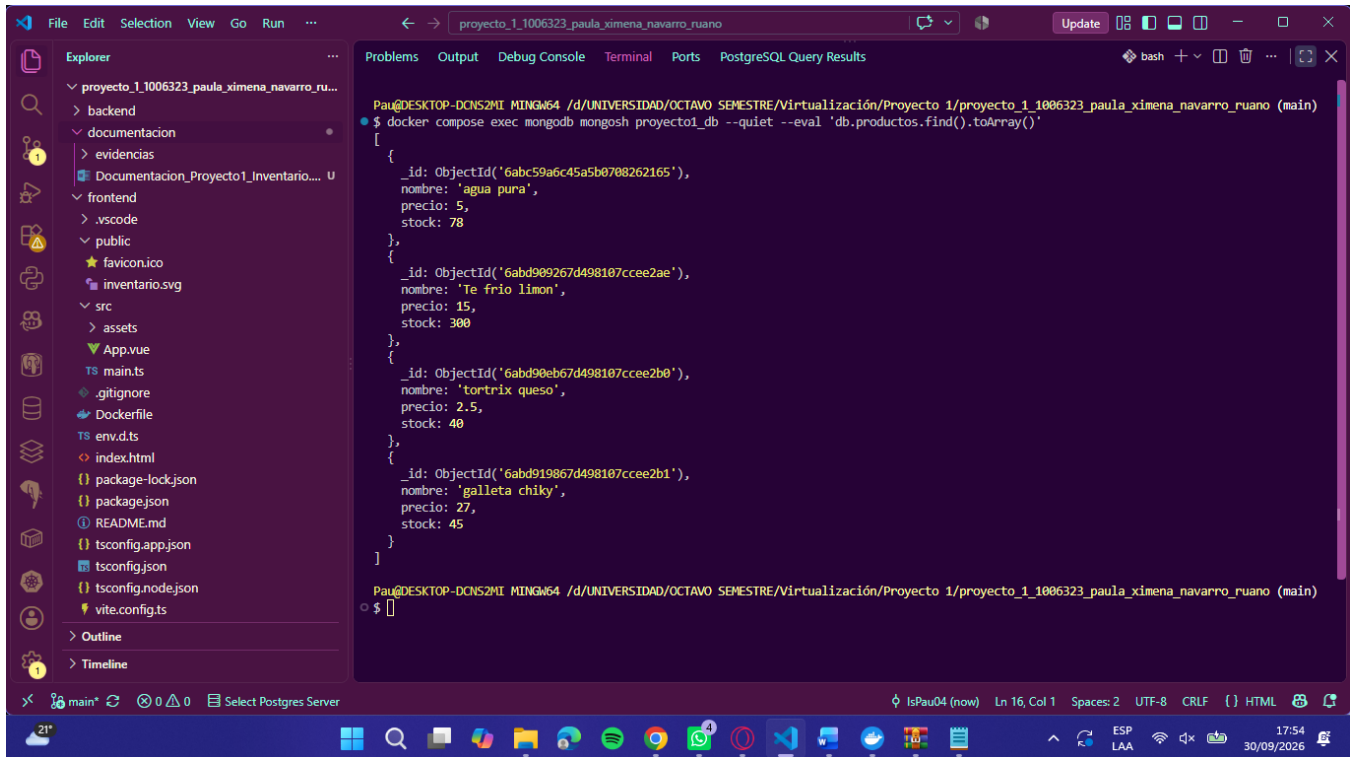
docker volume ls lista los volúmenes e inspect muestra la configuración de un volumen específico. La inspección de mongodb confirma proyecto1_mongo_data montado en /data/db. MongoDB también dispone de un montaje en /data/configdb; el volumen utilizado para los productos del proyecto es proyecto1_mongo_data.

Consulta directa de los documentos

```
docker compose exec mongodb mongosh proyecto1_db --quiet \
--eval 'db.productos.find().toArray()'
```

El comando ejecuta mongosh dentro del servicio MongoDB y lista los documentos de productos. La captura muestra agua pura, Te frio limon, tortrix queso y galleta chiky, con sus identificadores, precios y existencias, sin publicar el puerto de la base al host.

Documentos almacenados en MongoDB



The screenshot shows a VS Code editor with a file explorer on the left and a terminal window on the right. The file explorer shows a project structure with folders like 'backend', 'documentation', 'evidencias', 'frontend', 'public', 'src', and 'assets'. The terminal window shows the command 'docker compose exec mongodb mongosh proyecto1_db --quiet --eval 'db.productos.find().toArray()'' and the resulting JSON output of four product documents.

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose exec mongodb mongosh proyecto1_db --quiet --eval 'db.productos.find().toArray()'
[
  {
    _id: ObjectId('6abc59a6c45a5b0708262165'),
    nombre: 'agua pura',
    precio: 5,
    stock: 78
  },
  {
    _id: ObjectId('6abd90e67d498107ccee2ae'),
    nombre: 'Te frio limon',
    precio: 15,
    stock: 300
  },
  {
    _id: ObjectId('6abd919867d498107ccee2b0'),
    nombre: 'tortrix queso',
    precio: 2.5,
    stock: 40
  },
  {
    _id: ObjectId('6abd919867d498107ccee2b1'),
    nombre: 'galleta chiky',
    precio: 27,
    stock: 45
  }
]
```

Evidencia 5. Documentos almacenados en MongoDB.

Actividad: Consultar productos dentro de MongoDB.

Comando o configuración: docker compose exec mongodb mongosh proyecto1_db --quiet --eval 'db.productos.find().toArray()'

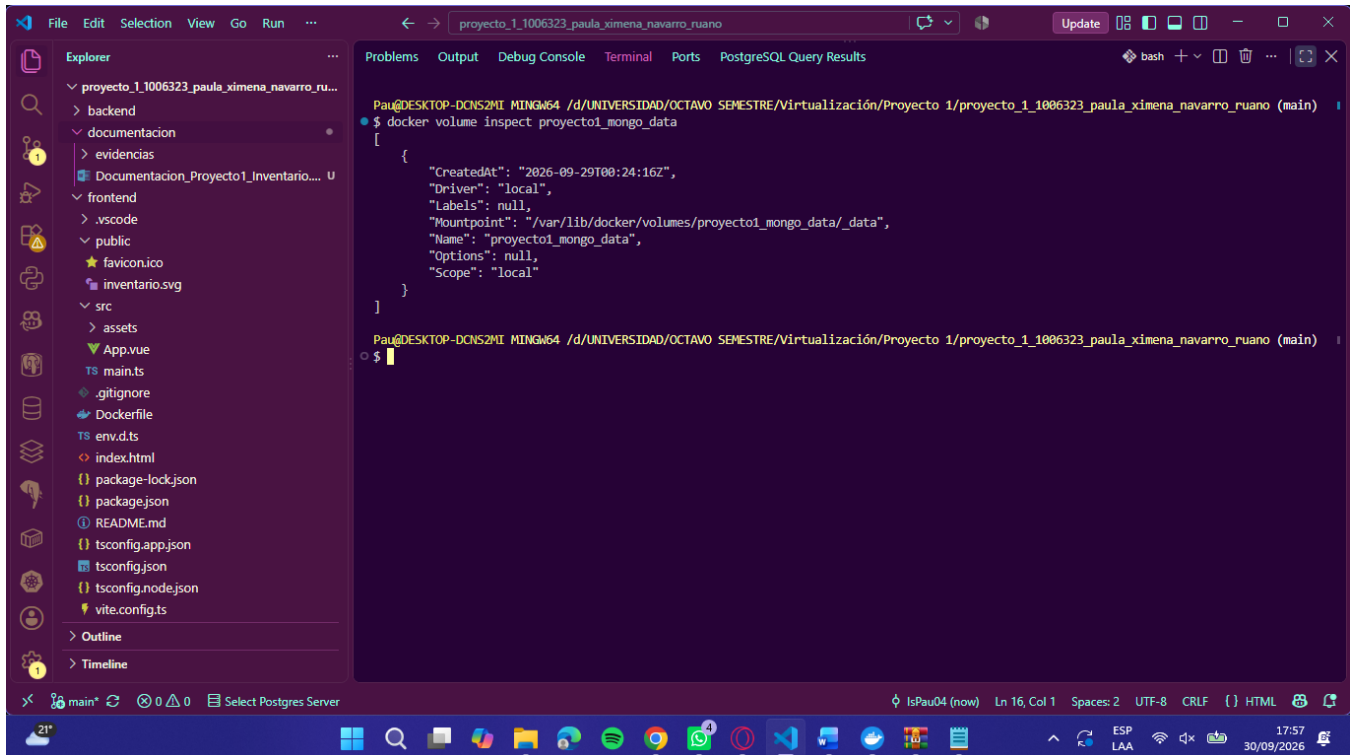
Funcionamiento: Ejecuta mongosh en el servicio y obtiene la colección productos.

Motivo: Verificar directamente el almacenamiento.

Resultado: Se muestran los cuatro productos con _id, nombre, precio y stock.

Qué demuestra: Los datos de la interfaz tienen documentos en la base.

Volumen persistente de MongoDB



```
File Edit Selection View Go Run ...
proyecto_1_1006323_paula_ximena_navarro_ruano
Update
Problems Output Debug Console Terminal Ports PostgreSQL Query Results
bash
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker volume inspect proyecto1_mongo_data
[
  {
    "CreatedAt": "2026-09-29T00:24:16Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/proyecto1_mongo_data/_data",
    "Name": "proyecto1_mongo_data",
    "Options": null,
    "Scope": "local"
  }
]
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$
```

Evidencia 6. Volumen persistente de MongoDB.

Actividad: Inspeccionar el recurso de almacenamiento.

Comando o configuración: `docker volume inspect proyecto1_mongo_data`

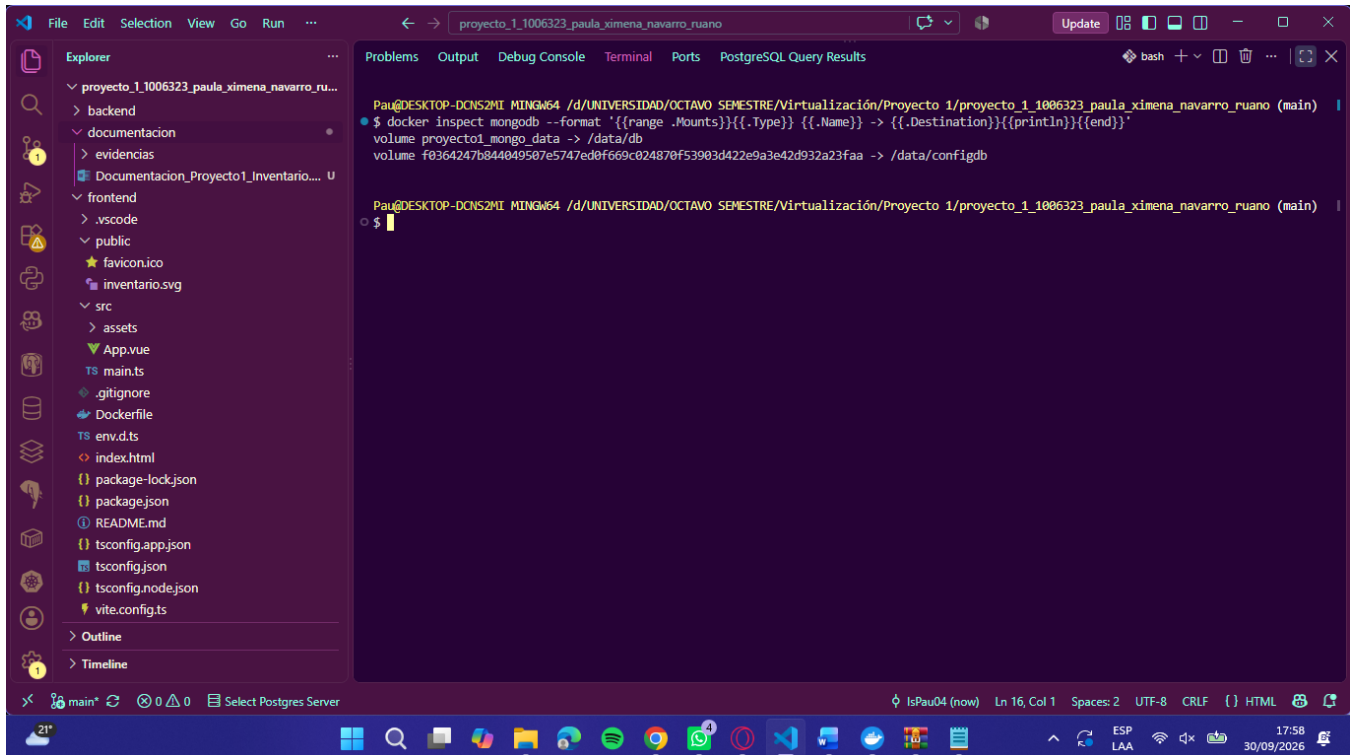
Funcionamiento: Consulta nombre, controlador y punto de montaje del volumen.

Motivo: Identificar dónde guarda Docker los datos persistentes.

Resultado: El volumen `proyecto1_mongo_data` usa el controlador local y tiene un punto de montaje dentro de Docker.

Qué demuestra: Existe un recurso de almacenamiento independiente del contenedor.

Montaje del volumen en MongoDB



Evidencia 7. Montaje del volumen en MongoDB.

Actividad: Revisar los montajes del contenedor.

Comando o configuración: `docker inspect mongodb --format '{{range .Mounts}}{{.Type}} {{.Name}} -> {{.Destination}}{{println}}{{end}}'`

Funcionamiento: Muestra el tipo, nombre y destino de cada montaje.

Motivo: Relacionar el volumen del proyecto con el directorio de datos.

Resultado: proyecto1_mongo_data aparece montado en /data/db.

Qué demuestra: MongoDB utiliza el volumen nombrado para sus archivos de datos.

Etapa 6 Variables de entorno

```
MONGO_HOST=mongodb
MONGO_PORT=27017
MONGO_DATABASE=proyecto1_db
BACKEND_HOST=0.0.0.0
BACKEND_PORT=5000
FLASK_DEBUG=false
HOST_BIND_IP=127.0.0.1
HTTP_PORT=8082
HTTPS_PORT=8443
DOCKER_NETWORK_NAME=proyecto_network
MONGO_VOLUME_NAME=proyecto1_mongo_data
NETWORK_SUBNET=192.168.162.0/27
NETWORK_GATEWAY=192.168.162.1
GATEWAY_IP=192.168.162.12
FRONTEND_IP=192.168.162.13
BACKEND_IP=192.168.162.14
MONGODB_IP=192.168.162.15
```

Este es el contenido de configuración previsto para `.env` y de referencia en `.env.example`. `MONGO_HOST` identifica al servicio MongoDB en la red Docker; `MONGO_PORT` indica su puerto interno; `MONGO_DATABASE` selecciona la base utilizada por Flask.

Compose lee `.env` para sustituir `${MONGO_HOST}`, `${MONGO_PORT}` y `${MONGO_DATABASE}`. La forma `${VARIABLE:?Falta ... en .env}` produce un error explícito si falta una variable o está vacía. `environment` transmite los valores al contenedor. Flask los recupera mediante `os.getenv`. Son dos momentos distintos: sustitución en Compose e interpretación en Python.

```
cp -n .env.example .env
docker compose config --quiet
```

`cp -n` crea `.env` desde el ejemplo si todavía no existe y evita sobrescribirlo. `docker compose config --quiet` valida el modelo y la interpolación sin imprimir la configuración resuelta. En la conversación, la validación terminó sin errores ni salida.

Verificación dentro del backend

```
docker compose exec backend python -c "import os; print({k: os.getenv(k) for k in ['MONGO_HOST', 'MONGO_PORT', 'MONGO_DATABASE', 'BACKEND_HOST', 'BACKEND_PORT', 'FLASK_DEBUG']})"
```

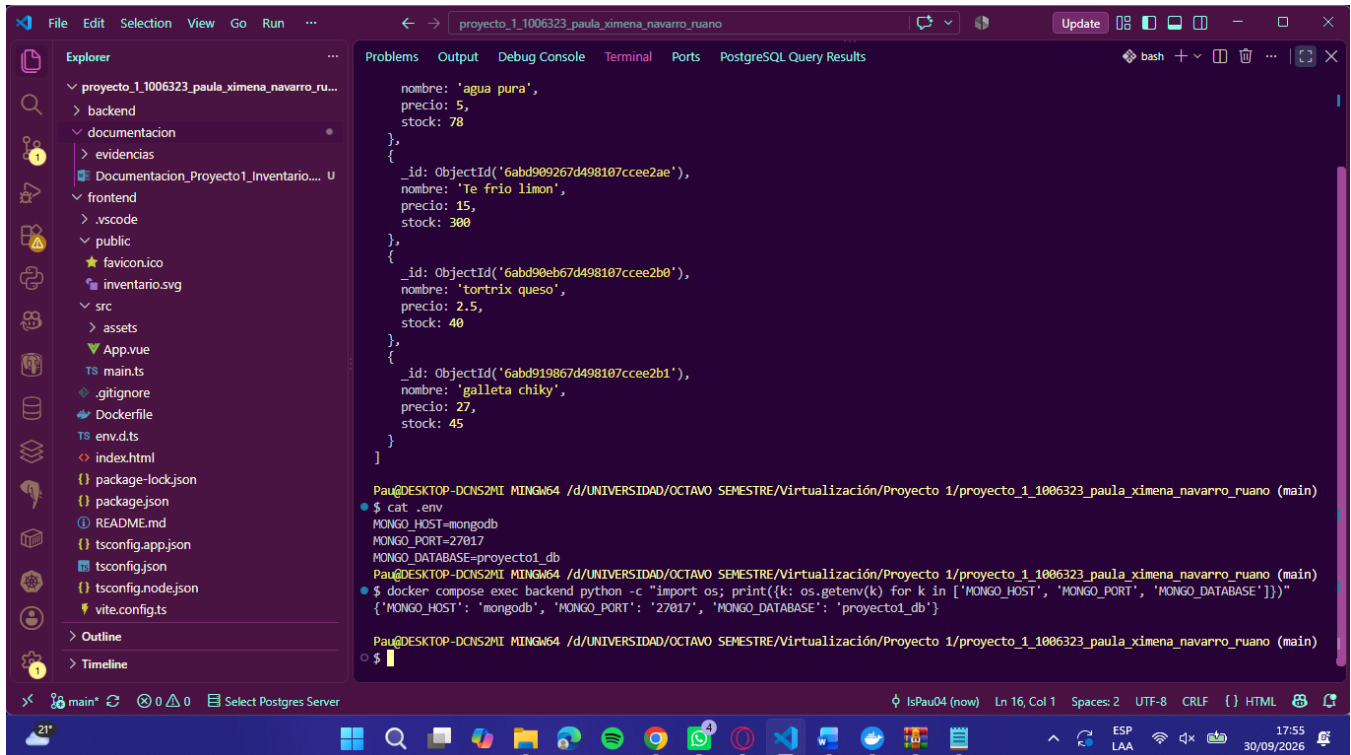
La captura consulta las tres variables utilizadas por el backend y compara el contenido de `.env` con los valores recibidos dentro del contenedor: `MONGO_HOST=mongodb`, `MONGO_PORT=27017` y `MONGO_DATABASE=proyecto1_db`.

Configuración y repositorio

`.env` se mantiene local y está excluido de Git. `.env.example` permite reconstruir los valores. La llave `nginx/certs/server.key` también está excluida mediante `nginx/certs/*.key`. No se debe pegar el contenido de la llave en las evidencias.

La configuración final separa la conexión a MongoDB, el host y puerto de Flask, la depuración, los puertos publicados, los nombres de red y volumen y las IP fijas mediante `.env`. `NETWORK_SUBNET` y `NETWORK_GATEWAY` se usan al crear la red externa; modificar el archivo no cambia automáticamente una red existente. Los puertos internos 80 y 443 de Nginx se mantienen como parte de su configuración de servicio.

Variables recibidas por el backend



The screenshot shows a VS Code editor window with the file explorer on the left and the terminal on the right. The file explorer shows a project structure with files like `App.vue`, `main.ts`, `Dockerfile`, and `env.d.ts`. The terminal panel displays the output of a Python script that reads environment variables and prints them as a JSON array.

```
nombre: 'agua pura',  
precio: 5,  
stock: 78  
},  
{  
  _id: ObjectId('6abd909267d498107ccee2ae'),  
  nombre: 'Te frio limon',  
  precio: 15,  
  stock: 300  
},  
{  
  _id: ObjectId('6abd90eb67d498107ccee2b0'),  
  nombre: 'tortrix queso',  
  precio: 2.5,  
  stock: 40  
},  
{  
  _id: ObjectId('6abd919867d498107ccee2b1'),  
  nombre: 'galleta chiky',  
  precio: 27,  
  stock: 45  
}  
]
```

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)  
$ cat .env  
MONGO_HOST=mongodb  
MONGO_PORT=27017  
MONGO_DATABASE=proyecto1_db  
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)  
$ docker compose exec backend python -c "import os; print({k: os.getenv(k) for k in ['MONGO_HOST', 'MONGO_PORT', 'MONGO_DATABASE']})"  
{'MONGO_HOST': 'mongodb', 'MONGO_PORT': '27017', 'MONGO_DATABASE': 'proyecto1_db'}  
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)  
$
```

Evidencia 8. Variables recibidas por el backend.

Actividad: Comparar variables locales y del contenedor.

Comando o configuración: `cat .env` y `docker compose exec backend python -c con os.getenv`

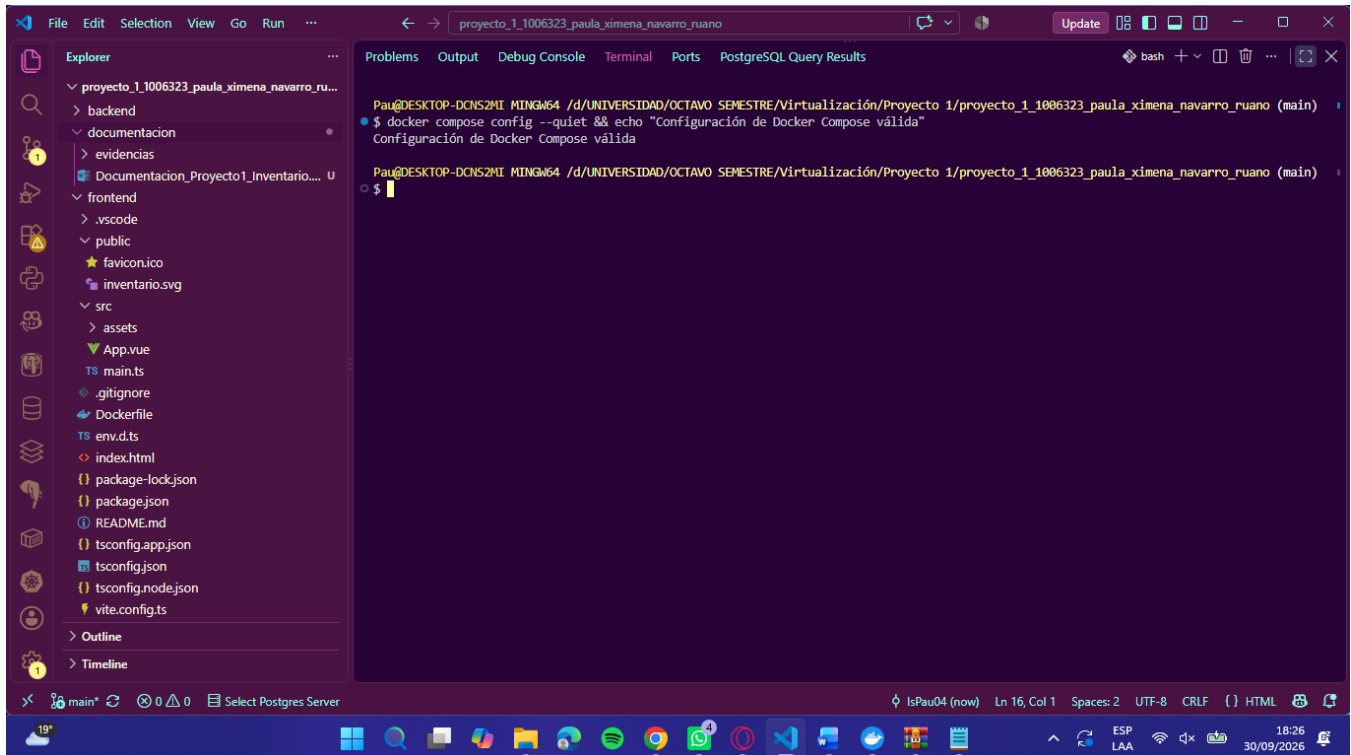
Funcionamiento: `cat` muestra `.env` y Python lee las tres variables de conexión.

Motivo: Confirmar la transmisión de configuración por Compose.

Resultado: Host mongodb, puerto 27017 y base proyecto1_db coinciden en ambas consultas.

Qué demuestra: Flask recibe los valores de conexión esperados.

Validación de Docker Compose



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose config --quiet && echo "Configuración de Docker Compose válida"
Configuración de Docker Compose válida

Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$
```

Evidencia 9. Validación de Docker Compose.

Actividad: Validar la configuración antes de ejecutarla.

Comando o configuración: `docker compose config --quiet && echo "Configuración de Docker Compose válida"`

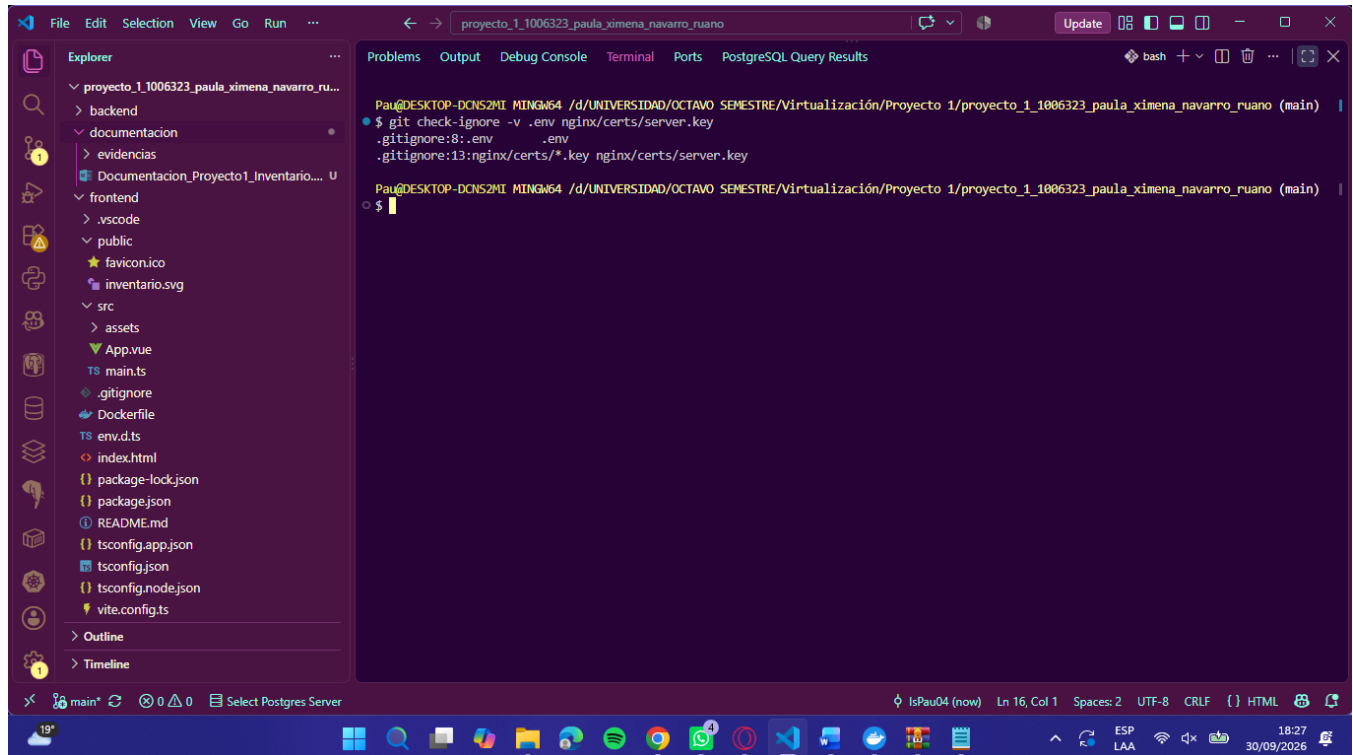
Funcionamiento: config comprueba el modelo; && muestra el mensaje únicamente si termina correctamente.

Motivo: Detectar errores de sintaxis o interpolación.

Resultado: Aparece Configuración de Docker Compose válida.

Qué demuestra: El archivo Compose y sus variables superan la validación.

Exclusión de archivos locales en Git



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$ git check-ignore -v .env nginx/certs/server.key
.gitignore:8:.env
.gitignore:13:nginx/certs/*.key nginx/certs/server.key

Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$
```

Evidencia 10. Exclusión de archivos locales en Git.

Actividad: Consultar las reglas de exclusión.

Comando o configuración: git check-ignore -v .env nginx/certs/server.key

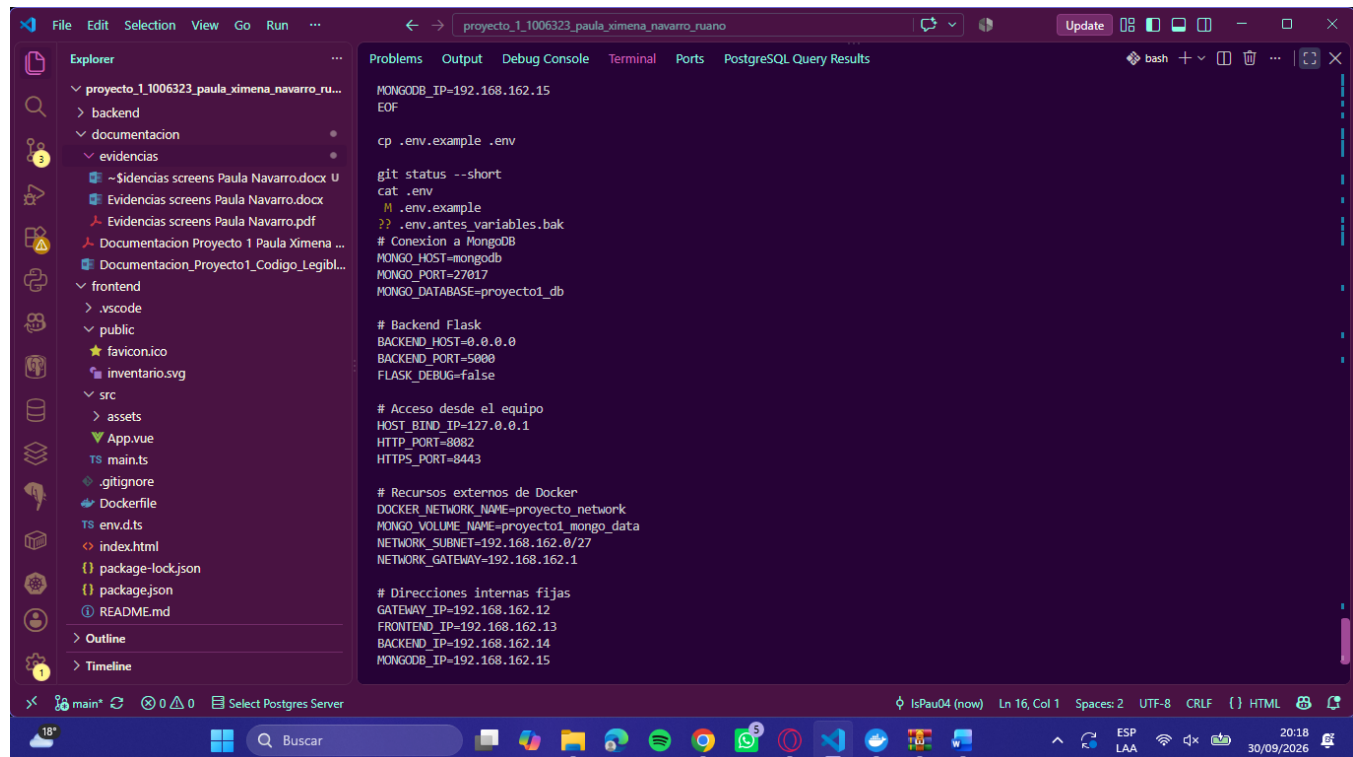
Funcionamiento: Git identifica el archivo .gitignore y la regla aplicable a cada ruta.

Motivo: Mantener las variables locales y la llave privada fuera del seguimiento normal.

Resultado: Se muestran las reglas .env y nginx/certs/*.key.

Qué demuestra: Ambas rutas están cubiertas por .gitignore.

Variables de entorno de la configuración final



```
projecto_1_1006323_paula_ximena_navarro_ruano
bash
MONGODB_IP=192.168.162.15
EOF
cp .env.example .env
git status --short
cat .env
M .env.example
?? .env.antes_variables.bak
# Conexion a MongoDB
MONGO_HOST=mongodb
MONGO_PORT=27017
MONGO_DATABASE=proyecto1_db

# Backend Flask
BACKEND_HOST=0.0.0.0
BACKEND_PORT=5000
FLASK_DEBUG=false

# Acceso desde el equipo
HOST_BIND_IP=127.0.0.1
HTTP_PORT=8082
HTTPS_PORT=8443

# Recursos externos de Docker
DOCKER_NETWORK_NAME=proyecto1_network
MONGO_VOLUME_NAME=proyecto1_mongo_data
NETWORK_SUBNET=192.168.162.0/27
NETWORK_GATEWAY=192.168.162.1

# Direcciones internas fijas
GATEWAY_IP=192.168.162.12
FRONTEND_IP=192.168.162.13
BACKEND_IP=192.168.162.14
MONGODB_IP=192.168.162.15
```

Evidencia 30. Variables de entorno de la configuración final.

Actividad: Separar los valores configurables de los servicios.

Comando o configuración: Actualización de .env.example y copia a .env; consulta con cat .env.

Funcionamiento: Compose interpola los valores y los entrega a los servicios declarados.

Motivo: Modificar parámetros sin editar la lógica de la aplicación.

Resultado: Se muestran los valores de MongoDB, Flask, puertos, recursos externos, subred, gateway e IP fijas.

Qué demuestra: La configuración final incluye variables más allá de la conexión a MongoDB.

Lectura de las variables desde Flask

```
if __name__ == "__main__":
    app.run(
        host=os.getenv("BACKEND_HOST", "0.0.0.0"),
        port=int(os.getenv("BACKEND_PORT", "5000")),
        debug=os.getenv("FLASK_DEBUG", "false").lower()
            in ("true", "1", "yes")
    )
```

os.getenv obtiene cada valor. int convierte el puerto a número. La comparación interpreta true, 1 o yes como activación de depuración; false la desactiva. La consulta dentro del backend devolvió MONGO_HOST=mongodb, MONGO_PORT=27017, MONGO_DATABASE=proyecto1_db, BACKEND_HOST=0.0.0.0, BACKEND_PORT=5000 y FLASK_DEBUG=false. Esa salida también aparece en la parte superior de la evidencia de generación del certificado.

Etapa 7 Red Docker y direccionamiento

Los cuatro servicios comparten proyecto_network, una red bridge externa creada previamente. Su subred comprobada es 192.168.162.0/27 y su gateway es 192.168.162.1. El gateway de la red Docker es distinto del contenedor gateway Nginx: el primero corresponde al encaminamiento de la red y el segundo al acceso web.

Servicio	Contenedor	IP fija	Puerto interno
gateway	proyecto1-gateway	192.168.162.12	80 y 443
frontend	proyecto1-frontend	192.168.162.13	80
backend	proyecto1-backend	192.168.162.14	5000
mongodb	mongodb	192.168.162.15	27017

Una máscara /27 corresponde a 255.255.255.224. El rango va de .0 a .31; .0 es la dirección de red y .31 la de broadcast. Las IP asignadas están dentro del rango utilizable. El nombre de servicio permite resolver destinos por DNS interno, aunque también se asignen IP fijas.

```
docker network ls
docker network inspect proyecto_network
docker network inspect proyecto_network --format '{{json .IPAM.Config}}'
```

El primer comando lista las redes. El segundo muestra IPAM y los contenedores conectados. El tercero extrae la subred y el gateway. La salida real conservada del tercero fue:

```
[{"Subnet": "192.168.162.0/27", "Gateway": "192.168.162.1"}]
```

Actividad: Comprobar el direccionamiento de la red final.

Comando o configuración: docker network inspect proyecto_network --format ...

Funcionamiento: Consulta la configuración IPAM de la red existente.

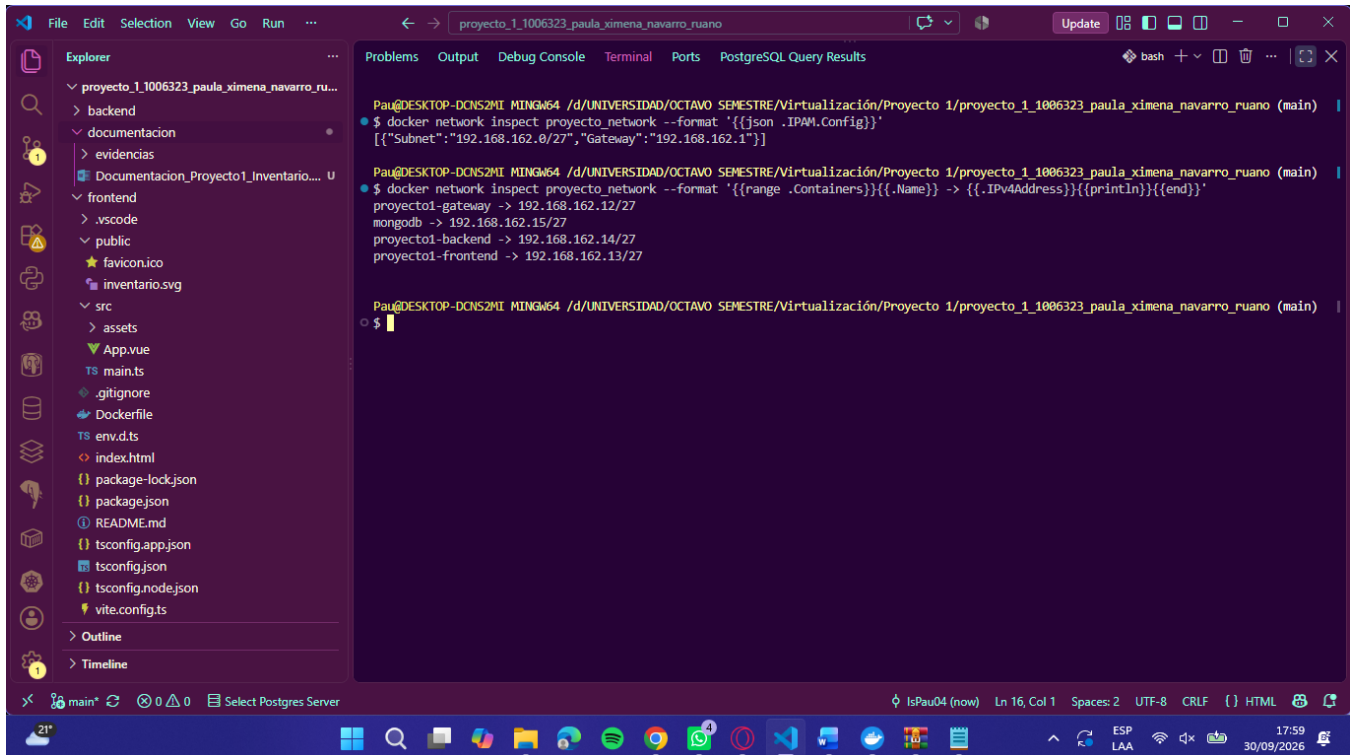
Motivo: Verificar que las direcciones estáticas de Compose son compatibles.

Resultado: subred 192.168.162.0/27, puerta de enlace 192.168.162.1 e IP de los cuatro contenedores, según la inspección capturada.

Qué demuestra: la configuración IPAM y las direcciones efectivamente asignadas coinciden con el direccionamiento definido para los servicios.

Para la comunicación entre contenedores utilizo los nombres de servicio mongodb, backend y frontend. Docker los resuelve dentro de la red compartida. localhost identifica al propio contenedor, por lo que solo se utiliza en comprobaciones internas del mismo servicio y no para conectar servicios diferentes.

Red y direcciones IP de los servicios



The screenshot shows a VS Code editor with a terminal window open. The terminal displays the output of two Docker commands. The first command, `docker network inspect proyecto_network --format '{{json .IPAM.Config}}'`, returns a JSON object for the network configuration: `{\"Subnet\": \"192.168.162.0/27\", \"Gateway\": \"192.168.162.1\"}`. The second command, `docker network inspect proyecto_network --format '{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{println}}{{end}}'`, lists the containers and their assigned IP addresses: `proyecto1-gateway -> 192.168.162.12/27`, `mongodb -> 192.168.162.15/27`, `proyecto1-backend -> 192.168.162.14/27`, and `proyecto1-frontend -> 192.168.162.13/27`. The VS Code interface shows a file explorer on the left with a project structure including `backend`, `documentation`, `evidencias`, `frontend`, `public`, `src`, and `assets`. The terminal window title is `proyecto_1_1006323_paula_ximena_navarro_ruano`.

Evidencia 11. Red y direcciones IP de los servicios.

Actividad: Inspeccionar direccionamiento y contenedores conectados.

Comando o configuración: `docker network inspect proyecto_network` con los formatos de IPAM y Containers

Funcionamiento: Extrae subred, puerta de enlace y dirección IPv4 de cada contenedor.

Motivo: Comparar la red real con Compose.

Resultado: Subred 192.168.162.0/27; gateway .1; Nginx .12, frontend .13, backend .14 y MongoDB .15.

Qué demuestra: Los cuatro servicios comparten la red y tienen las IP previstas.

Etaapa 8 Docker Compose

docker-compose.yml declara name: proyecto1-supermercado y los servicios mongodb, backend, frontend y gateway. MongoDB y Nginx usan imágenes de registro; backend y frontend tienen build y generan imágenes propias. Los cuatro usan restart: unless-stopped. Los fragmentos siguientes resumen las secciones relevantes; el archivo completo incluye las validaciones \${VARIABLE:?Falta VARIABLE en .env} y los healthchecks descritos.

Servicio MongoDB

```
mongodb:
  image: mongo:8.0
  container_name: mongodb
  restart: unless-stopped
  command: ["mongod", "--bind_ip_all", "--port", "${MONGO_PORT}"]
  environment:
    MONGO_INITDB_DATABASE: ${MONGO_DATABASE}
    MONGO_PORT: "${MONGO_PORT}"
  volumes:
    - mongo_data:/data/db
  networks:
    proyecto_network:
      ipv4_address: ${MONGODB_IP}
```

image elige la imagen base y container_name fija el nombre. environment pasa la base de inicialización. volumes conecta el almacenamiento. networks asigna una IP en la red compartida. La política de reinicio ayuda a recuperar contenedores después de reiniciar Docker, salvo que se hayan detenido explícitamente.

El healthcheck de MongoDB ejecuta mongosh con --port tomado de MONGO_PORT y evalúa quit(db.adminCommand({ping: 1}).ok ? 0 : 1). Un ping correcto produce código cero. Se verifica cada 10 segundos, con timeout de 5 segundos, 5 reintentos y start_period de 20 segundos. El servicio inicia mongod con el mismo puerto, por lo que la prueba y la conexión de Flask permanecen coordinadas.

Servicio backend

```
backend:
  build: ./backend
  image: proyecto1-backend:1.0
  container_name: proyecto1-backend
  restart: unless-stopped
  environment:
    MONGO_HOST: ${MONGO_HOST}
    MONGO_PORT: "${MONGO_PORT}"
    MONGO_DATABASE: ${MONGO_DATABASE}
    BACKEND_HOST: ${BACKEND_HOST}
    BACKEND_PORT: "${BACKEND_PORT}"
    FLASK_DEBUG: "${FLASK_DEBUG}"
  depends_on:
    mongodb:
      condition: service_healthy
  networks:
    proyecto_network:
      ipv4_address: ${BACKEND_IP}
```

build identifica la carpeta del Dockerfile e image asigna la etiqueta local. environment entrega a Flask las seis variables de conexión y ejecución. depends_on espera que MongoDB esté healthy. El healthcheck del backend construye la URL con os.environ["BACKEND_PORT"] y consulta /api/productos en su propio localhost, con timeout de 3 segundos. Se conserva interval de 10 segundos, timeout de 5 segundos, 5 reintentos y start_period de 15 segundos.

Etaapa 8 Frontend gateway y recursos externos

```
frontend:
  build: ./frontend
  image: proyecto1-frontend:1.0
  container_name: proyecto1-frontend
  restart: unless-stopped
  networks:
    proyecto_network:
      ipv4_address: ${FRONTEND_IP}

gateway:
  image: nginx:alpine
  container_name: proyecto1-gateway
  restart: unless-stopped
  environment:
    BACKEND_PORT: "${BACKEND_PORT}"
    HTTPS_PORT: "${HTTPS_PORT}"
  ports:
    - "${HOST_BIND_IP}:${HTTP_PORT}:80"
    - "${HOST_BIND_IP}:${HTTPS_PORT}:443"
  volumes:
    - type: bind
      source: ./nginx/nginx.conf.template
      target: /etc/nginx/templates/default.conf.template
      read_only: true
      bind:
        create_host_path: false
    - type: bind
      source: ./nginx/certs
      target: /etc/nginx/certs
      read_only: true
      bind:
        create_host_path: false
  depends_on:
    backend:
      condition: service_healthy
    frontend:
      condition: service_started
  networks:
    proyecto_network:
      ipv4_address: ${GATEWAY_IP}
```

El frontend no publica puertos y no tiene un healthcheck explícito. El gateway espera al backend saludable y al frontend iniciado; `service_started` no garantiza una comprobación funcional de la página. Los bind mounts permiten leer la configuración y los certificados del host. `read_only` impide modificarlos desde el gateway y `create_host_path: false` evita crear rutas ausentes automáticamente.

```
networks:
  proyecto_network:
    external: true
    name: ${DOCKER_NETWORK_NAME}
volumes:
  mongo_data:
    external: true
    name: ${MONGO_VOLUME_NAME}
```

`external: true` indica que Compose reutiliza recursos ya existentes y no los crea al levantar servicios. El nombre lógico `mongo_data` se vincula al nombre real `proyecto1_mongo_data`. El archivo completo entregado en el repositorio contiene además los healthchecks explicados en la página anterior.

Etapla 9 Gateway Nginx y proxy inverso

```
server {
    listen 80;
    server_name localhost 127.0.0.1;
    return 301 https://$host:${HTTPS_PORT}$request_uri;
}

server {
    listen 443 ssl;
    server_name localhost 127.0.0.1;
    ssl_certificate /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    location /api/ {
        proxy_pass http://backend:${BACKEND_PORT};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    location / {
        proxy_pass http://frontend:80;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

La configuración activa del gateway es `nginx/nginx.conf.template`. Contiene bloques `server` y se monta en `/etc/nginx/templates/default.conf.template`. Al iniciar, la imagen oficial de Nginx sustituye `BACKEND_PORT` y `HTTPS_PORT` y genera `/etc/nginx/conf.d/default.conf`, incluido dentro del bloque `http` de la configuración principal. Por eso la plantilla no contiene `events` ni un bloque `http` adicional. El antiguo `nginx/nginx.conf` queda como referencia histórica y no se monta en el Compose actual.

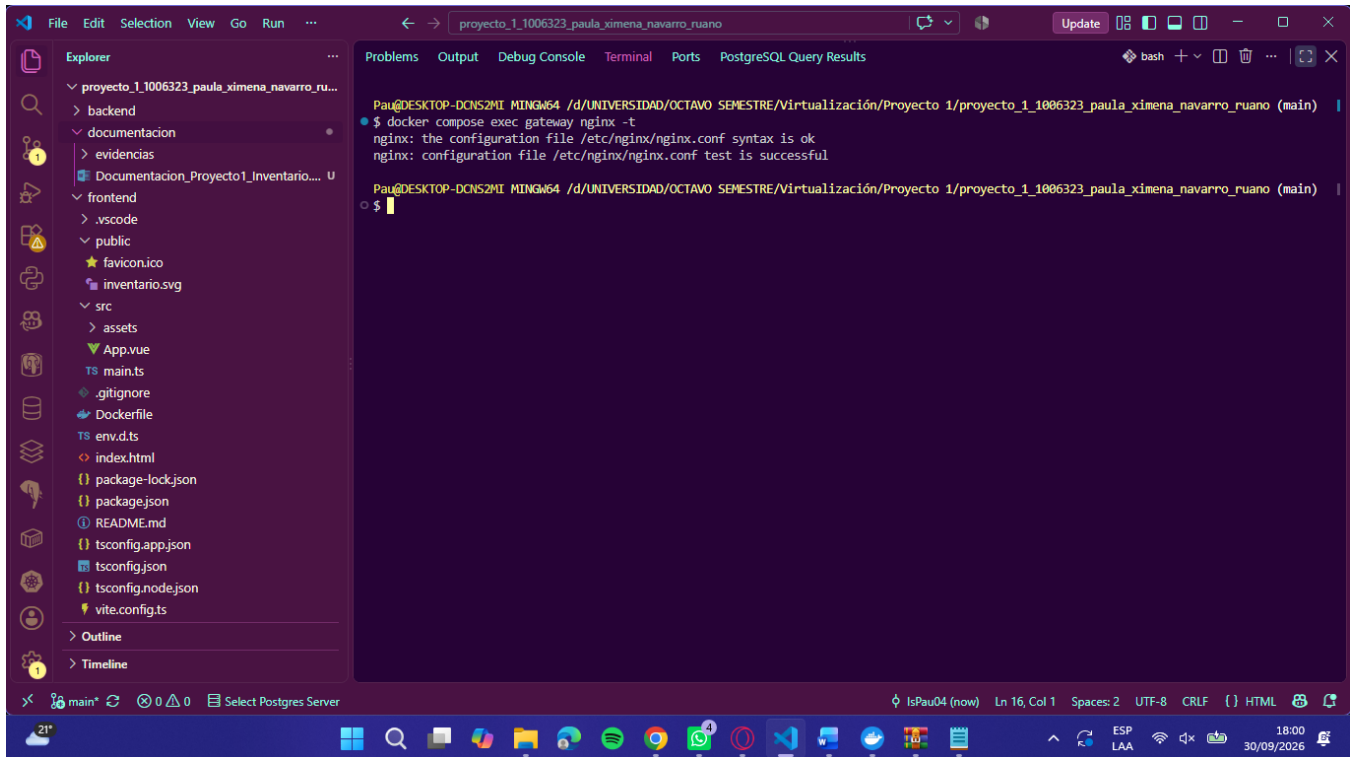
El segundo `server` escucha en 443 con TLS. `ssl_certificate` y `ssl_certificate_key` apuntan al certificado y a su llave dentro del directorio montado. `ssl_protocols` habilita TLS 1.2 y 1.3. El primer `server` escucha en 80 y redirige hacia `https://$host:${HTTPS_PORT}$request_uri`. En la configuración generada se observa 8443, conservando el `host` y la ruta solicitada.

`location /api/` envía solicitudes a `backend:${BACKEND_PORT}`; con el valor actual, `backend:5000`. `proxy_pass` conserva `/api/productos` al no añadir una barra final al `upstream`. `location /` envía el resto hacia `frontend:80`. Docker resuelve ambos nombres dentro de la red. `proxy_set_header` conserva datos de la solicitud; `$host`, `$remote_addr`, `$scheme` y `$request_uri` permanecen como variables de Nginx.

```
docker compose exec gateway nginx -t
```

`nginx -t` valida la sintaxis y los archivos requeridos dentro del gateway activo. La evidencia final devuelve `syntax is ok` y `test is successful`. La captura histórica posterior conserva la verificación inicial del enrutamiento y se identifica como una etapa de desarrollo.

Validación del gateway Nginx final



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |  
$ docker compose exec gateway nginx -t  
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok  
nginx: configuration file /etc/nginx/nginx.conf test is successful  
  
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |  
$
```

Evidencia 12. Validación del gateway Nginx final.

Actividad: Probar la configuración del gateway.

Comando o configuración: docker compose exec gateway nginx -t

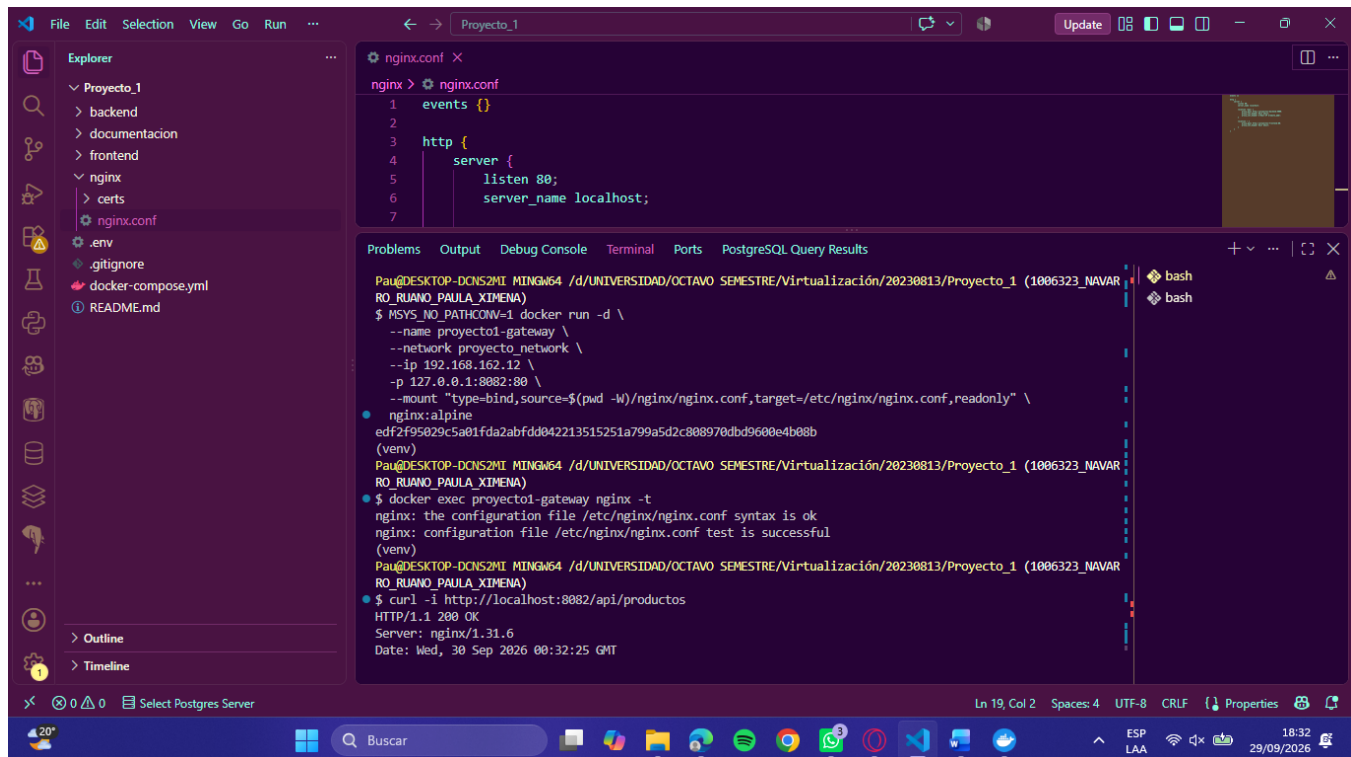
Funcionamiento: Nginx analiza su configuración y comprueba los archivos requeridos.

Motivo: Detectar errores antes de realizar las pruebas web.

Resultado: syntax is ok y test is successful.

Qué demuestra: La configuración final supera la validación dentro del contenedor.

Eta9a 9 Evidencia inicial del enrutamiento



The screenshot shows the Visual Studio Code interface with the following components:

- Explorer:** Shows the project structure for 'Proyecto_1', including 'backend', 'documentation', 'frontend', 'nginx', 'nginx.conf', '.env', '.gitignore', 'docker-compose.yml', and 'README.md'.
- Editor:** Displays the 'nginx.conf' file with the following configuration:

```
1 events {}
2
3 http {
4     server {
5         listen 80;
6         server_name localhost;
7     }
8 }
```
- Problems:** Shows a list of errors and warnings, including 'nginx: the configuration file /etc/nginx/nginx.conf syntax is ok' and 'nginx: configuration file /etc/nginx/nginx.conf test is successful'.
- Terminal:** Displays the execution of Docker commands and the output of a curl request:

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/20230813/Proyecto_1 (1006323_NAVAR
RO_RUANO_PAULA_XIMENA)
$ MSYS_NO_PATHCONV=1 docker run -d \
  --name proyecto1-gateway \
  --network proyecto_network \
  --ip 192.168.162.12 \
  -p 127.0.0.1:8882:80 \
  --mount "type=bind,source=$(pwd -W)/nginx/nginx.conf,target=/etc/nginx/nginx.conf,readonly" \
  nginx:alpine
edf2f95029c5a01fda2abfdd042213515251a799a5d2c808970dbd9600e4b08b
(venv)
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/20230813/Proyecto_1 (1006323_NAVAR
RO_RUANO_PAULA_XIMENA)
$ docker exec proyecto1-gateway nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
(venv)
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/20230813/Proyecto_1 (1006323_NAVAR
RO_RUANO_PAULA_XIMENA)
$ curl -i http://localhost:8882/api/productos
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Wed, 30 Sep 2026 00:32:25 GMT
```

Figura 7. Prueba histórica del gateway HTTP durante el desarrollo.

Actividad: Verificar la configuración inicial de Nginx y el acceso a la API por proxy.

Comando o configuración: `docker exec proyecto1-gateway nginx -t` y `curl -i http://localhost:8882/api/productos`.

Funcionamiento: El primero prueba la configuración; el segundo solicita la API a través del gateway.

Motivo: Comprobar el enrutamiento antes de incorporar HTTPS y la orquestación final.

Resultado: Nginx informó `syntax is ok` y `test is successful`; la petición devolvió `HTTP/1.1 200 OK`.

Qué demuestra: Acredita la integración inicial. La captura muestra `docker run`, que fue una prueba preparatoria; la solución entregada se levanta con Docker Compose.

Montajes comprobados antes de la parametrización

La inspección posterior del gateway confirmó que sus montajes apuntan a la carpeta final del proyecto, no a la antigua carpeta del repositorio del curso. Se observaron `nginx/nginx.conf` → `/etc/nginx/nginx.conf` y `nginx/certs` → `/etc/nginx/certs`. Estos resultados se conservan como transcripción, no como captura original.

El proxy inverso concentra la entrada web y oculta las direcciones internas a la interfaz. Si Flask falla, Nginx puede devolver un error de upstream y el frontend mostrará el problema al consultar. Si Nginx falla, no habrá acceso web aunque MongoDB y Flask sigan activos.

Comprobación de la plantilla y del despliegue final

```
MSYS_NO_PATHCONV=1 docker compose exec gateway \  
cat /etc/nginx/conf.d/default.conf
```

MSYS_NO_PATHCONV=1 evita que Git Bash convierta la ruta Linux a una ruta de Windows. La salida verificada conservó \$host y \$request_uri, reemplazó HTTPS_PORT por 8443 y BACKEND_PORT por 5000, y mostró proxy_pass hacia backend y frontend. Después de reconstruir, docker compose ps mostró los cuatro servicios activos, MongoDB y backend healthy, nginx -t fue correcto, HTTP devolvió 301 y la API HTTPS devolvió 200 con los cuatro productos. Estos resultados corresponden a las salidas finales de terminal compartidas durante la actualización.

Etaapa 10 Certificado y llave privada

Generé un certificado autofirmado para localhost y una llave RSA de 2048 bits. El archivo openssl.cnf proporciona la identidad y las extensiones necesarias para que el certificado contemple tanto localhost como 127.0.0.1.

```
[req]
prompt = no
distinguished_name = identidad
x509_extensions = extensiones
[identidad]
CN = localhost
[extensiones]
subjectAltName = DNS:localhost,IP:127.0.0.1
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature,keyEncipherment
extendedKeyUsage = serverAuth
```

Generación desde la raíz del proyecto

```
openssl req \
  -config nginx/certs/openssl.cnf \
  -x509 -nodes -days 365 -newkey rsa:2048 -sha256 \
  -keyout nginx/certs/server.key \
  -out nginx/certs/server.crt
```

req procesa la identidad y genera el material solicitado. -config elige explícitamente el archivo de configuración. -x509 emite un certificado autofirmado; -newkey rsa:2048 genera la llave; -sha256 utiliza ese algoritmo de firma; -days 365 fija la vigencia. -nodes deja la llave sin contraseña para que Nginx pueda iniciar automáticamente. -keyout y -out determinan sus archivos de salida.

prompt = no evita preguntas interactivas. subjectAltName establece las identidades válidas. CA:FALSE indica un certificado de servidor, no una autoridad certificadora. keyUsage y extendedKeyUsage limitan sus usos previstos. server.crt es público; server.key es privado y debe corresponder al certificado.

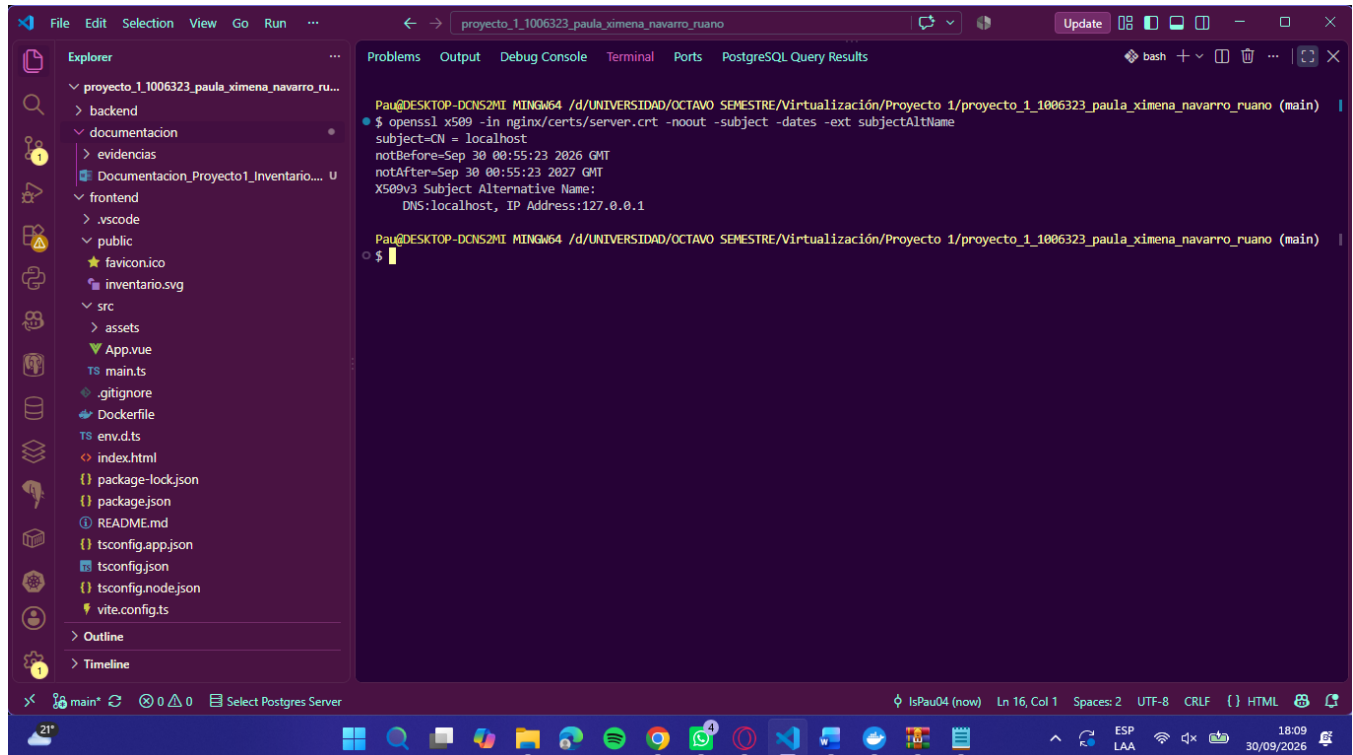
Comprobación

```
openssl x509 -in nginx/certs/server.crt \
  -noout -subject -dates -ext subjectAltName
```

Este comando muestra identidad, vigencia y nombres alternativos sin imprimir el certificado completo. La captura inicial conserva el certificado anterior; la nueva evidencia documenta el par regenerado con CN=localhost, SAN localhost y 127.0.0.1 y vigencia del 01/10/2026 al 01/10/2027 en UTC. La regeneración ocurrió el 30 de septiembre en horario de Guatemala. Después se validó y recargó Nginx y se comprobó la API por HTTPS.

HTTPS usa TLS para proteger confidencialidad e integridad del tráfico y comprobar identidad mediante certificados. HTTP no proporciona esa protección. El certificado vincula la identidad con una llave pública; la llave privada permite al servidor acreditar que le pertenece. Un certificado autofirmado cifra la conexión, pero su emisor no es de confianza automática para el navegador.

Identidad y vigencia del certificado inicial



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$ openssl x509 -in nginx/certs/server.crt -noout -subject -dates -ext subjectAltName
subject=CN = localhost
notBefore=Sep 30 00:55:23 2026 GMT
notAfter=Sep 30 00:55:23 2027 GMT
X509v3 Subject Alternative Name:
    DNS:localhost, IP Address:127.0.0.1

Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$
```

Evidencia 13. Identidad y vigencia del certificado inicial.

Actividad: Inspeccionar el certificado utilizado.

Comando o configuración: openssl x509 -in nginx/certs/server.crt -noout -subject -dates -ext subjectAltName

Funcionamiento: Lee sujeto, fechas de validez y nombres alternativos.

Motivo: Comprobar su correspondencia con el acceso local.

Resultado: CN=localhost; SAN DNS:localhost e IP:127.0.0.1; vigencia del 30 de septiembre de 2026 al 30 de septiembre de 2027.

Qué demuestra: El certificado contiene los nombres usados para acceder al gateway.

Generación del certificado TLS y llave privada RSA

Evidencia 31. Generación del certificado TLS y llave privada RSA.

Actividad: Regenerar el certificado autofirmado y su llave después de respaldar el par anterior.

Comando o configuración: `openssl req -config nginx/certs/openssl.cnf -x509 -nodes -days 365 -newkey rsa:2048 -sha256 -keyout nginx/certs/server.key -out nginx/certs/server.crt`

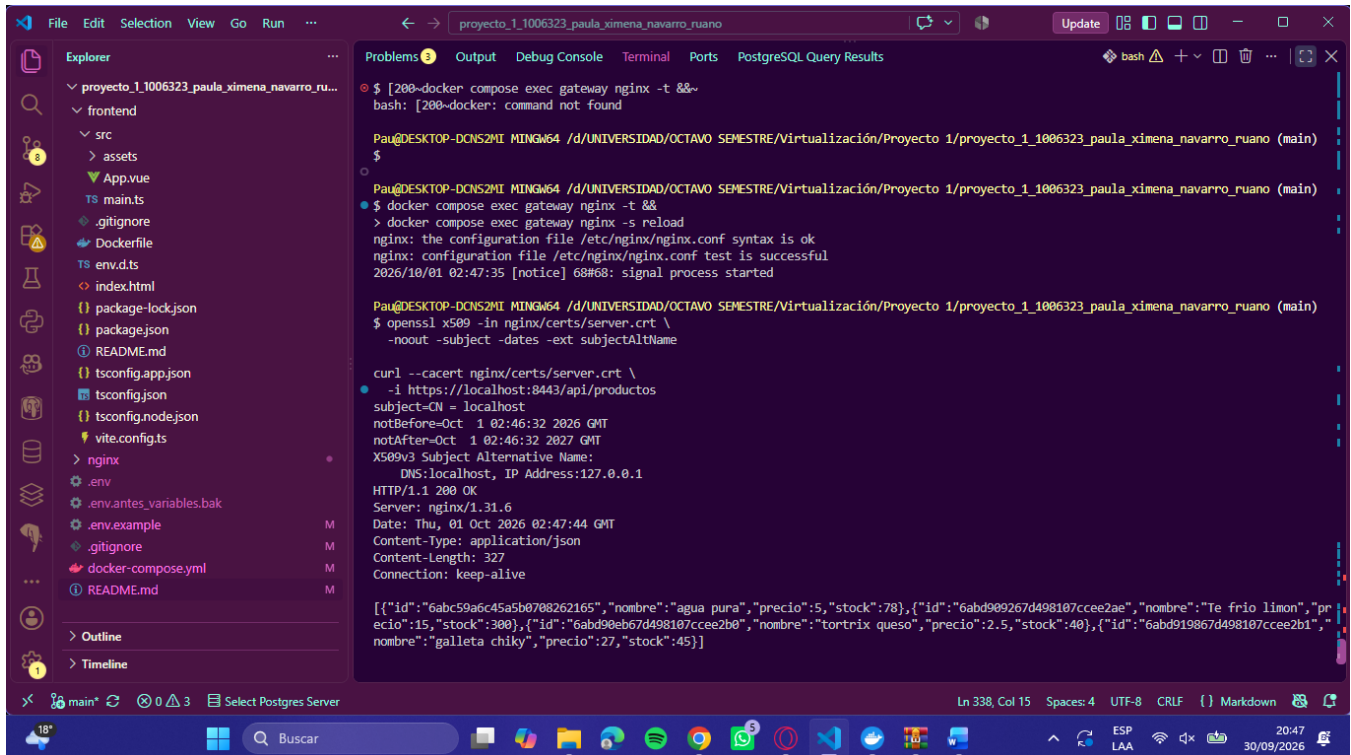
Funcionamiento: Genera una llave RSA de 2048 bits y un certificado autofirmado válido por 365 días con SHA-256.

Motivo: Documentar el proceso requerido por el enunciado y obtener un par correspondiente.

Resultado: OpenSSL muestra el progreso y regresa al prompt después de ejecutar el comando.

Qué demuestra: La generación se ejecutó desde la carpeta final del proyecto.

Verificación del certificado regenerado y acceso HTTPS



```
File Edit Selection View Go Run ...
projecto_1_1006323_paula_ximena_navarro_ruano
Update

Explorer
projecto_1_1006323_paula_ximena_navarro_ruano
  frontend
    src
      assets
      App.vue
      main.ts
    .gitignore
    Dockerfile
    env.d.ts
    index.html
    package-lock.json
    package.json
    README.md
    tsconfig.app.json
    tsconfig.json
    tsconfig.node.json
    vite.config.ts
  nginx
  .env
  .env antes variables.bak
  .env.example
  .gitignore
  docker-compose.yml
  README.md

Problems Output Debug Console Terminal Ports PostgreSQL Query Results
bash
$ [200-docker compose exec gateway nginx -t &&~
bash: [200-docker: command not found

Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose exec gateway nginx -t &&
> docker compose exec gateway nginx -s reload
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/10/01 02:47:35 [notice] 68#68: signal process started

Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ openssl x509 -in nginx/certs/server.crt \
-noout -subject -dates -ext subjectAltName

curl --cacert nginx/certs/server.crt \
-i https://localhost:8443/api/productos
subject=CN = localhost
notBefore=Oct 1 02:46:32 2026 GMT
notAfter=Oct 1 02:46:32 2027 GMT
X509v3 Subject Alternative Name:
DNS:localhost, IP Address:127.0.0.1
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Thu, 01 Oct 2026 02:47:44 GMT
Content-Type: application/json
Content-Length: 327
Connection: Keep-alive

[{"id":"6abc59a6c45a5b0708262165","nombre":"agua pura","precio":5,"stock":78}, {"id":"6abd989267d498187ccee2ae","nombre":"Te frio limon","pr
ecio":15,"stock":300}, {"id":"6abd98eb57d498187ccee2b0","nombre":"tortrix queso","precio":2.5,"stock":40}, {"id":"6abd919867d498187ccee2b1","
nombre":"galleta chiky","precio":27,"stock":45}]

Ln 338, Col 15 Spaces: 4 UTF-8 CRLF {} Markdown
```

Evidencia 32. Verificación del certificado regenerado y acceso HTTPS.

Actividad: Validar y recargar Nginx, inspeccionar el certificado y consultar la API.

Comando o configuración: nginx -t; nginx -s reload; openssl x509 -noout -subject -dates -ext subjectAltName; curl --cacert nginx/certs/server.crt -i https://localhost:8443/api/productos.

Funcionamiento: Nginx valida y recarga el par generado; OpenSSL muestra identidad y vigencia; curl verifica TLS y solicita productos.

Motivo: Confirmar que el nuevo certificado se utiliza sin perder el acceso a los datos.

Resultado: Nginx indica syntax is ok y test is successful; CN=localhost, SAN localhost/127.0.0.1, vigencia 01/10/2026–01/10/2027 UTC y HTTP/1.1 200 OK con cuatro productos.

Qué demuestra: El certificado regenerado es aceptado por curl con la referencia local y el gateway sigue conectando la API con MongoDB.

Etapas 11 y 12 Reconstrucción y ejecución

Los siguientes pasos permiten instalar el código final en otra computadora. Todos los comandos posteriores a `cd` se ejecutan en la raíz del proyecto con Docker Desktop iniciado. Las creaciones de red y volumen se realizan solamente si dichos recursos todavía no existen.

```
git clone https://github.com/IsPau04/proyecto_1_1006323_paula_ximena_navarro_ruano.git
cd proyecto_1_1006323_paula_ximena_navarro_ruano
cp -n .env.example .env
docker network ls
docker volume ls
```

`git clone` descarga el código y su historial. `cd` selecciona la carpeta. `cp` crea la configuración local. Los dos listados permiten comprobar los recursos externos antes de crearlos.

```
set -a
source .env
set +a
# Ejecutar create solamente si el recurso no existe.
docker network create --driver bridge \
  --subnet "$NETWORK_SUBNET" --gateway "$NETWORK_GATEWAY" \
  "$DOCKER_NETWORK_NAME"
docker volume create "$MONGO_VOLUME_NAME"
```

`network create` establece nombre, controlador, subred y gateway. `volume create` prepara el almacenamiento nombrado. Si la red ya existe, reviso su `inspect` para confirmar que coincida. En una computadora distinta, el código no transporta los datos del volumen anterior.

Luego genero el certificado y la llave con el comando de la etapa 10. La llave no se descarga de GitHub porque está excluida. Un certificado y una llave preexistentes, vigentes y correspondientes entre sí pueden reutilizarse.

```
docker compose config --quiet
docker compose up -d --build --wait
docker compose ps
docker images
```

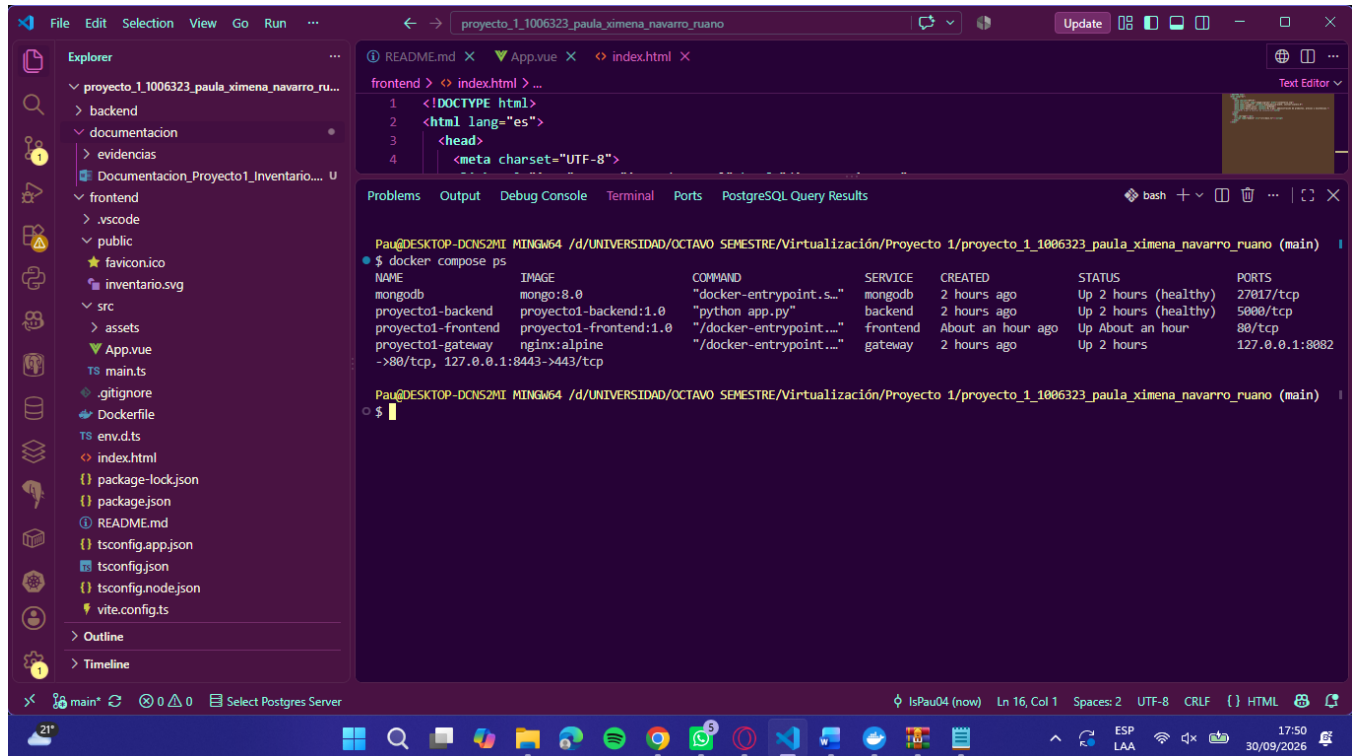
`config --quiet` valida. `up` crea e inicia la infraestructura; `-d` mantiene la ejecución en segundo plano; `--build` construye las imágenes antes de iniciar; `--wait` espera que los servicios estén iniciados o saludables según sus comprobaciones. `ps` muestra el estado y los puertos. `images` identifica las imágenes propias y las descargadas.

Resultado conservado

La construcción registrada generó `proyecto1-backend:1.0` y `proyecto1-frontend:1.0`. El comando utilizado durante la migración agregó `--force-recreate` para sustituir los contenedores existentes y sus montajes. En `ps`, MongoDB y backend aparecieron `healthy`; frontend y gateway aparecieron `Up`. No se interpreta `Up` como prueba de salud cuando no hay `healthcheck`.

Contenedor	Imagen	Estado registrado	Puertos
mongodb	mongo:8.0	Up healthy	27017/tcp interno
proyecto1-backend	proyecto1-backend:1.0	Up healthy	5000/tcp interno
proyecto1-frontend	proyecto1-frontend:1.0	Up	80/tcp interno
proyecto1-gateway	nginx:alpine	Up	127.0.0.1:8082→80 y 8443→443

Contenedores y puertos de la solución



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mongodb	mongo:8.0	"docker-entrypoint.s..."	mongodb	2 hours ago	Up 2 hours (healthy)	27017/tcp
proyecto1-backend	proyecto1-backend:1.0	"python app.py"	backend	2 hours ago	Up 2 hours (healthy)	5000/tcp
proyecto1-frontend	proyecto1-frontend:1.0	"/docker-entrypoint..."	frontend	About an hour ago	Up About an hour	80/tcp
proyecto1-gateway	nginx:alpine	"/docker-entrypoint..."	gateway	2 hours ago	Up 2 hours	127.0.0.1:8082->80/tcp, 127.0.0.1:8443->443/tcp

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$
```

Evidencia 14. Contenedores y puertos de la solución.

Actividad: Consultar el estado de los servicios.

Comando o configuración: docker compose ps

Funcionamiento: Lista contenedores, imágenes, estado y puertos publicados.

Motivo: Comprobar el despliegue completo.

Resultado: Los cuatro contenedores están activos; MongoDB y backend aparecen healthy. El gateway publica 8082 y 8443 en 127.0.0.1.

Qué demuestra: La solución está distribuida en cuatro servicios y solo el gateway publica acceso web.

Etapa 13 CRUD desde la interfaz Vue

La prueba principal se realiza en <https://localhost:8443>. Cada operación atraviesa Vue → Nginx → Flask → MongoDB. curl y mongosh sirven para complementar el diagnóstico, pero no sustituyen las evidencias de la interfaz exigidas por el ingeniero.

Operación	Acción en Vue	Trabajo de Flask y MongoDB
Crear	Ingresar nombre, precio y stock; guardar	POST → insert_one → respuesta 201 con ID
Consultar	Abrir página o pulsar Actualizar	GET → find → respuesta 200 con lista
Modificar	Editar, cambiar datos y guardar	PUT con ID → update_one/\$set → respuesta 200
Eliminar	Pulsar eliminar y confirmar	DELETE con ID → delete_one → respuesta 200

Consulta y actualización del inventario

Las figuras 4 y 5 muestran registros consultados desde Vue. La figura 4 conserva el mensaje de actualización exitosa. La figura 5 muestra los cuatro registros y sus acciones. El valor total observado es consistente con $Q5 \times 78 + Q15 \times 300 + Q2.50 \times 40 + Q27 \times 45 = Q6,205.00$; el stock suma 463 unidades.

Crear y eliminar

Registré Prueba documentación con precio Q10.00 y stock 5. La interfaz mostró el mensaje de registro correcto y el producto en la tabla. Después actualicé el stock a 8, manteniendo el precio, y al terminar la prueba de persistencia eliminé únicamente este producto mediante el diálogo de confirmación.

Secuencia de pruebas realizada

La secuencia fue crear el producto de prueba, consultar la tabla, modificar el stock, eliminar y recrear los contenedores, comprobar que el registro seguía disponible y finalmente eliminar el producto. Las capturas de esta etapa muestran la creación, la consulta del inventario, la modificación, la confirmación de eliminación y su resultado.

Con el producto de prueba registrado, la interfaz mostró 5 productos, 468 unidades y Q6,255.00. Después de cambiar el stock a 8, mostró 471 unidades y Q6,285.00. Al eliminar el producto, regresó a 4 productos, 463 unidades y Q6,205.00. Esta comparación respalda el efecto de las operaciones sobre el inventario.

Consulta del inventario desde Vue

The screenshot shows a web browser window with the URL `https://localhost:8443`. The application is titled "Mi Súper - Control de inventario". The main section is "Inventario de productos", which includes a sub-header "MÓDULO DE ALMACÉN" and a description "Administra tus productos, precios y existencias en un solo lugar."

Three summary cards are displayed:

- Productos registrados:** 4
- Unidades disponibles:** 463
- Valor del inventario:** Q6,205.00

Below the summary cards, there is a form to "Agregar producto" and a table of "Productos registrados".

Agregar producto form:

- Nombre del producto: Ej. Arroz blanco 1 lb
- Precio unitario: Q 0.00
- Stock disponible: 0 unidades

Productos registrados table:

PRODUCTO	PRECIO UNITARIO	STOCK	ACCIONES
A agua pura	Q5.00	78 unidades	[Edit] [Delete]
T Te frio limon	Q15.00	300 unidades	[Edit] [Delete]
T tortris queso	Q2.50	40 unidades	[Edit] [Delete]
G galleta chiky	Q27.00	45 unidades	[Edit] [Delete]

Evidencia 15. Consulta del inventario desde Vue.

Actividad: Abrir el sistema y consultar los productos.

Comando o configuración: `https://localhost:8443`

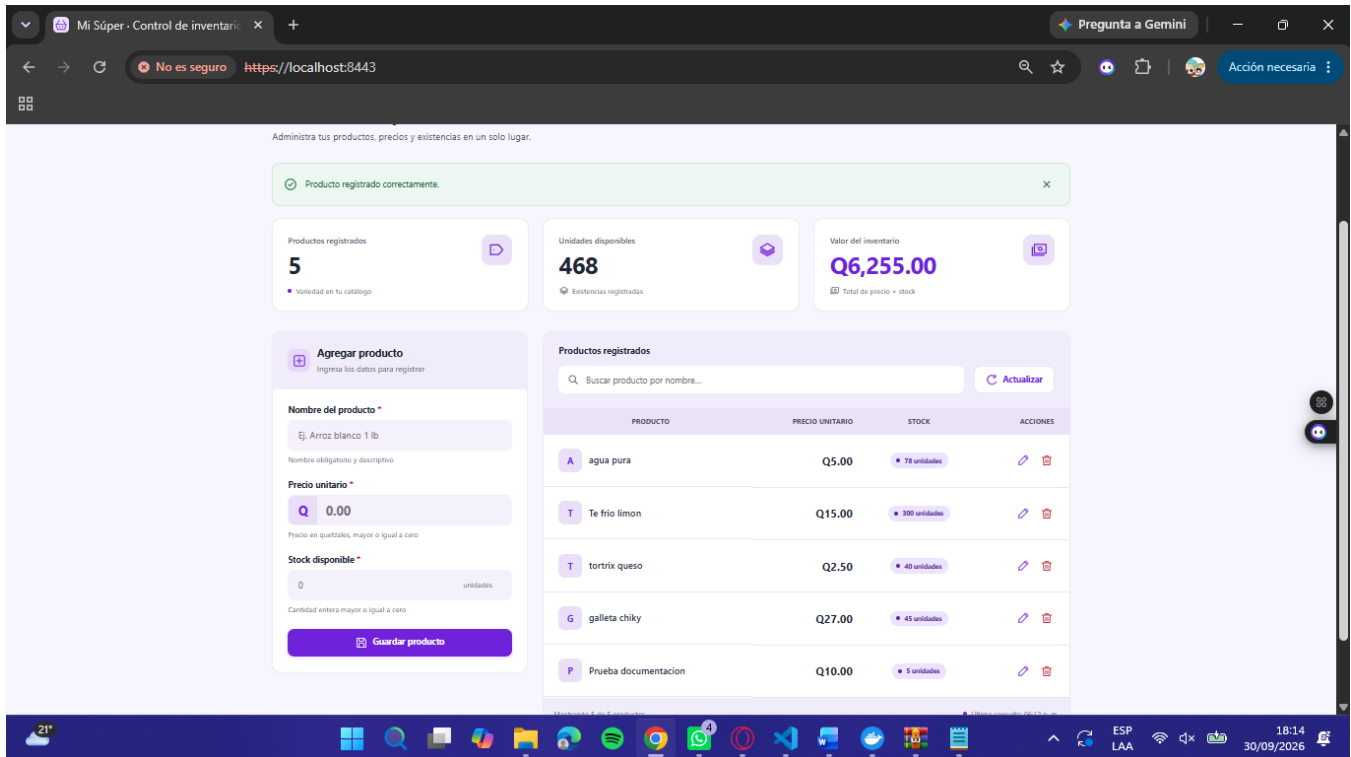
Funcionamiento: Vue solicita la lista por GET y presenta los datos devueltos por Flask.

Motivo: Verificar la lectura desde la interfaz.

Resultado: La tabla muestra los cuatro productos y los indicadores 463 unidades y Q6,205.00.

Qué demuestra: La interfaz recibe y presenta datos del inventario.

Creación de un producto desde Vue



Evidencia 16. Creación de un producto desde Vue.

Actividad: Registrar Prueba documentación a Q10.00 con stock 5.

Comando o configuración: Formulario de registro en <https://localhost:8443>

Funcionamiento: Vue envía POST; Nginx lo dirige a Flask, que inserta el documento en MongoDB.

Motivo: Verificar la operación de creación.

Resultado: Producto registrado correctamente; 5 productos, 468 unidades y Q6,255.00.

Qué demuestra: El nuevo producto se incorpora al inventario desde el frontend.

Actualización del stock desde Vue

The screenshot shows a web application interface for inventory management. At the top, there are three summary cards: 'Productos registrados' (5), 'Unidades disponibles' (471), and 'Valor del inventario' (Q6,285.00). Below these, on the left, is a form to 'Agregar producto' with fields for 'Nombre del producto', 'Precio unitario' (0.00), and 'Stock disponible' (0). On the right, a table lists 'Productos registrados' with columns for 'PRODUCTO', 'PRECIO UNITARIO', 'STOCK', and 'ACCIONES'. The table contains five items: 'agua pura' (Q5.00, 78 unidades), 'Te frio limon' (Q15.00, 300 unidades), 'tortrix queso' (Q2.50, 40 unidades), 'galleta chiky' (Q27.00, 45 unidades), and 'Prueba documentacion' (Q10.00, 8 unidades). The bottom of the page shows a Windows taskbar with the date 30/09/2026 and time 18:16.

PRODUCTO	PRECIO UNITARIO	STOCK	ACCIONES
A agua pura	Q5.00	78 unidades	[editar] [eliminar]
T Te frio limon	Q15.00	300 unidades	[editar] [eliminar]
T tortrix queso	Q2.50	40 unidades	[editar] [eliminar]
G galleta chiky	Q27.00	45 unidades	[editar] [eliminar]
P Prueba documentacion	Q10.00	8 unidades	[editar] [eliminar]

Evidencia 17. Actualización del stock desde Vue.

Actividad: Cambiar el stock de 5 a 8 y conservar el precio Q10.00.

Comando o configuración: Formulario de edición del producto de prueba

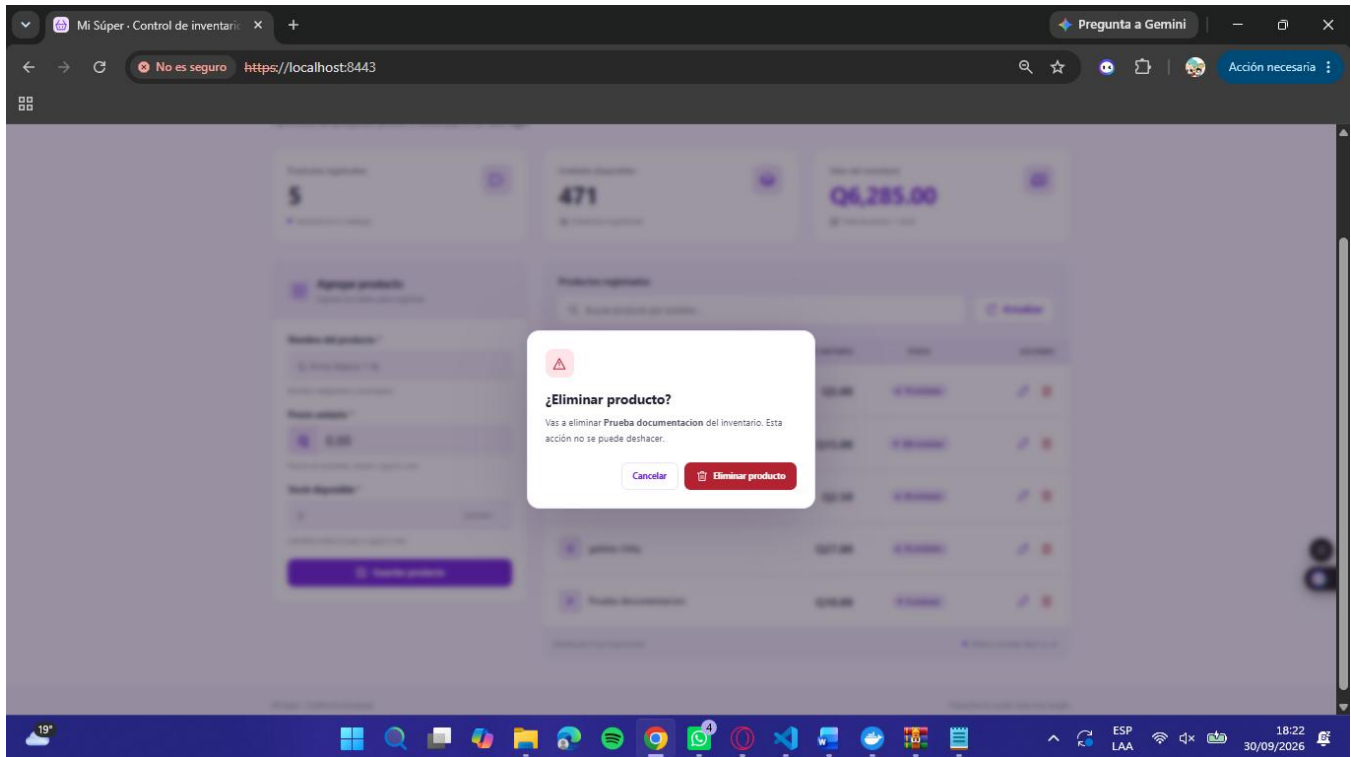
Funcionamiento: Vue envía PUT con el ID; Flask actualiza MongoDB y la interfaz consulta la lista.

Motivo: Verificar una modificación observable.

Resultado: El registro muestra stock 8; el resumen cambia a 471 unidades y Q6,285.00.

Qué demuestra: La modificación queda reflejada en la tabla y los indicadores.

Confirmación de eliminación desde Vue



Evidencia 18. Confirmación de eliminación desde Vue.

Actividad: Seleccionar la eliminación del producto temporal.

Comando o configuración: Diálogo Eliminar producto

Funcionamiento: El diálogo solicita confirmación antes de enviar DELETE.

Motivo: Evitar eliminar un producto por una pulsación accidental.

Resultado: La ventana identifica Prueba documentación y ofrece cancelar o eliminar.

Qué demuestra: La interfaz incorpora confirmación previa.

Resultado de eliminación del producto

Módulo de Almacén

Inventario de productos

Administra tus productos, precios y existencias en un solo lugar.

Producto eliminado correctamente.

Productos registrados

4

Verificar en tu catálogo

Unidades disponibles

463

Existencias registradas

Valor del inventario

Q6,205.00

Total de precio x stock

Agregar producto

Ingresa los datos para registrar

Nombre del producto *

Ej. Arroz blanco 1 lb

Nombre obligatorio y descriptivo

Precio unitario *

Q 0.00

Precio en quetzales, mayor o igual a cero

Stock disponible *

0 unidades

Cantidad entera mayor o igual a cero

Guardar producto

Productos registrados

Buscar producto por nombre...

Actualizar

PRODUCTO	PRECIO UNITARIO	STOCK	ACCIONES
A agua pura	Q5.00	78 unidades	Editar Eliminar
T Te frio limon	Q15.00	300 unidades	Editar Eliminar
T tortris queso	Q2.50	40 unidades	Editar Eliminar
G galleta chiky	Q27.00	45 unidades	Editar Eliminar

Evidencia 19. Resultado de eliminación del producto.

Actividad: Eliminar únicamente el producto temporal al finalizar la persistencia.

Comando o configuración: Confirmación de eliminación de Prueba documentación

Funcionamiento: Vue envía DELETE por Nginx; Flask elimina el documento y se recarga la lista.

Motivo: Verificar el resultado final de la operación.

Resultado: Producto eliminado correctamente; el registro desaparece y vuelven 4 productos, 463 unidades y Q6,205.00.

Qué demuestra: La eliminación se refleja en el inventario.

Etapa 14 Persistencia de los productos

La persistencia se basa en el volumen proyecto1_mongo_data montado en /data/db. Compose lo declara externo, por lo que docker compose down elimina los contenedores del proyecto y conserva este recurso externo. Al levantar nuevamente MongoDB, el servicio vuelve a montar los mismos datos.

Prueba de recreación de contenedores

```
docker compose down
docker compose up -d --wait
docker compose ps
```

down detiene y elimina los contenedores administrados por Compose. up los crea nuevamente y --wait espera su disponibilidad. ps comprueba que todos estén activos. Esta prueba no requiere eliminar volúmenes ni cambiar el nombre del recurso.

Antes de ejecutar down, la captura de actualización muestra Prueba documentación con precio Q10.00 y stock 8. Después de levantar los contenedores y recargar Vue, el mismo nombre, precio y stock siguen disponibles, sin registrar nuevamente el producto.

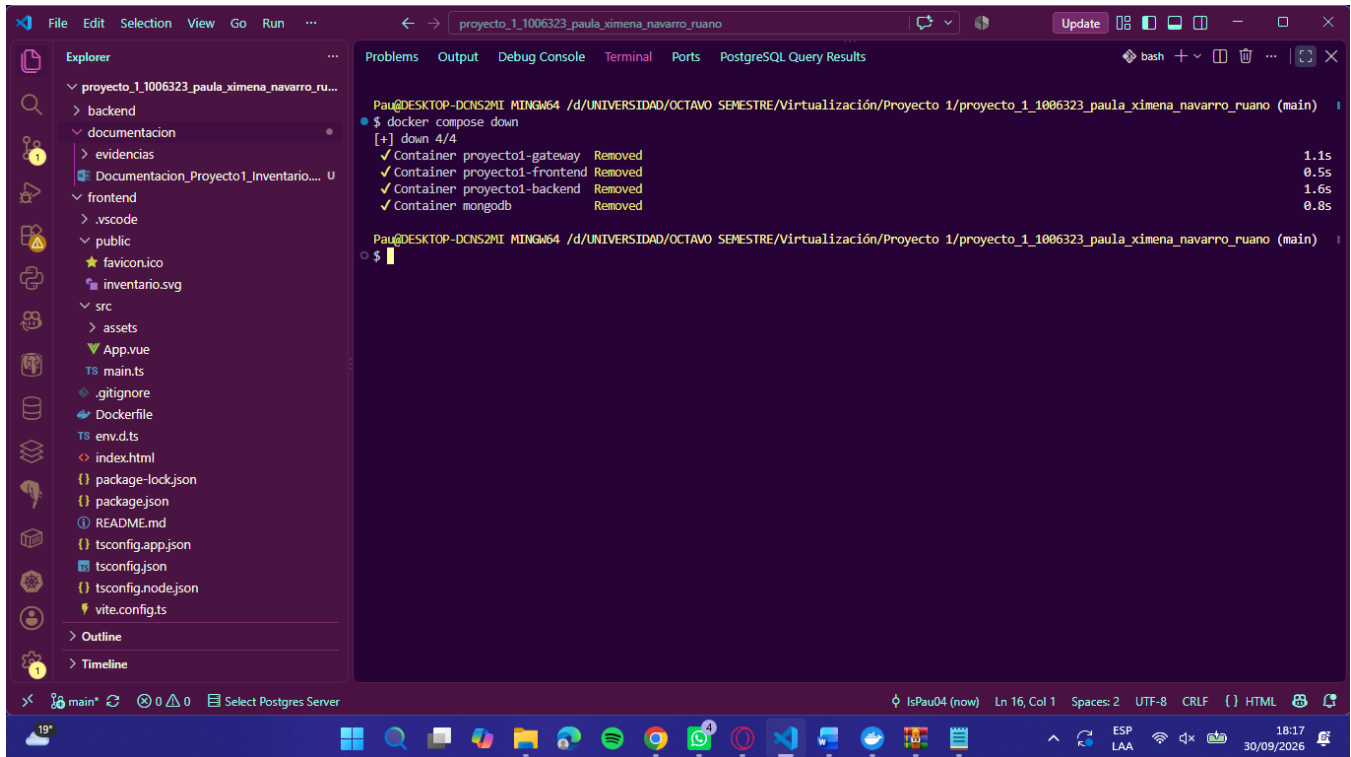
Estado de la evidencia

La secuencia capturada muestra los cuatro contenedores eliminados mediante down, su puesta en marcha con up -d --wait, el estado posterior con ps y el producto todavía disponible en Vue. La captura previa de la etapa 13 y la captura posterior de esta etapa conservan 5 productos, 471 unidades y Q6,285.00, con el producto de prueba a Q10.00 y stock 8.

Captura necesaria	Contenido que debe verse
Antes	Producto de prueba con precio y stock
Detención	docker compose down concluido
Reinicio	docker compose up -d --wait y ps
Después	Mismo producto y valores en Vue
Volumen	docker volume ls y montaje /data/db

El volumen pertenece al entorno Docker de esa computadora. Reiniciar el equipo o recrear los contenedores conserva los datos si el volumen permanece. Clonar el repositorio en otro equipo genera una instalación nueva y no copia automáticamente la base existente.

Detención y eliminación de los contenedores



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose down
[+] down 4/4
✓ Container proyecto1-gateway Removed 1.15
✓ Container proyecto1-frontend Removed 0.55
✓ Container proyecto1-backend Removed 1.65
✓ Container mongodb Removed 0.85

Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$
```

Evidencia 20. Detención y eliminación de los contenedores.

Actividad: Retirar los contenedores conservando los datos.

Comando o configuración: docker compose down

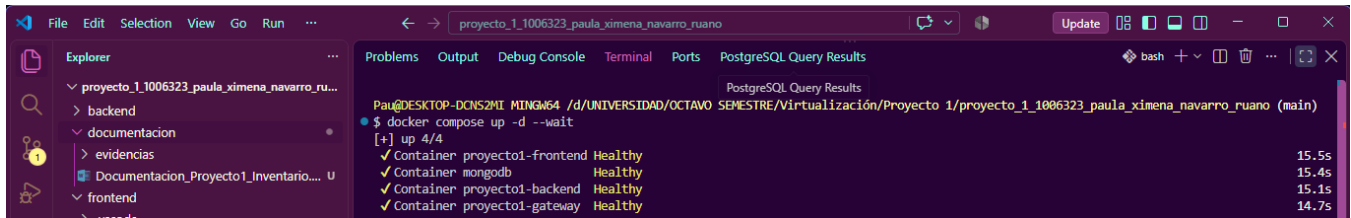
Funcionamiento: Compose detiene y elimina los cuatro servicios; conserva el volumen externo.

Motivo: Probar la independencia entre contenedores y almacenamiento.

Resultado: gateway, frontend, backend y mongodb aparecen Removed.

Qué demuestra: La prueba elimina los contenedores anteriores.

Puesta en marcha después de la detención



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose up -d --wait
[+] up 4/4
✓ Container proyecto1-frontend Healthy 15.5s
✓ Container mongodb Healthy 15.4s
✓ Container proyecto1-backend Healthy 15.1s
✓ Container proyecto1-gateway Healthy 14.7s
```

Evidencia 21. Puesta en marcha después de la detención.

Actividad: Crear e iniciar nuevamente los servicios.

Comando o configuración: docker compose up -d --wait

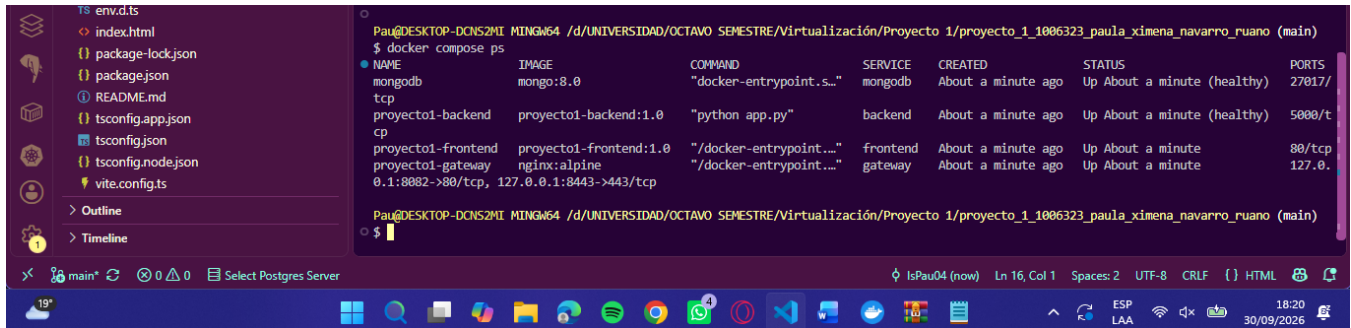
Funcionamiento: up recrea contenedores, -d ejecuta en segundo plano y --wait espera su disponibilidad.

Motivo: Recuperar la aplicación usando el mismo volumen.

Resultado: El comando termina con los cuatro contenedores iniciados.

Qué demuestra: La infraestructura puede recrearse mediante Compose.

Estado después de recrear los contenedores



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mongodb	mongo:8.0	"docker-entrypoint.s..."	mongodb	About a minute ago	Up About a minute (healthy)	27017/
tcp	proyecto1-backend:1.0	"python app.py"	backend	About a minute ago	Up About a minute (healthy)	5000/t
proyecto1-backend	proyecto1-backend:1.0	"python app.py"	backend	About a minute ago	Up About a minute (healthy)	5000/t
cp	proyecto1-frontend:1.0	"/docker-entrypoint..."	frontend	About a minute ago	Up About a minute	80/tcp
proyecto1-frontend	proyecto1-frontend:1.0	"/docker-entrypoint..."	frontend	About a minute ago	Up About a minute	80/tcp
proyecto1-gateway	nginx:alpine	"/docker-entrypoint..."	gateway	About a minute ago	Up About a minute	127.0.
proyecto1-gateway	0.1:8082->80/tcp, 127.0.0.1:8443->443/tcp					

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$
```

Evidencia 22. Estado después de recrear los contenedores.

Actividad: Consultar el estado posterior al inicio.

Comando o configuración: docker compose ps

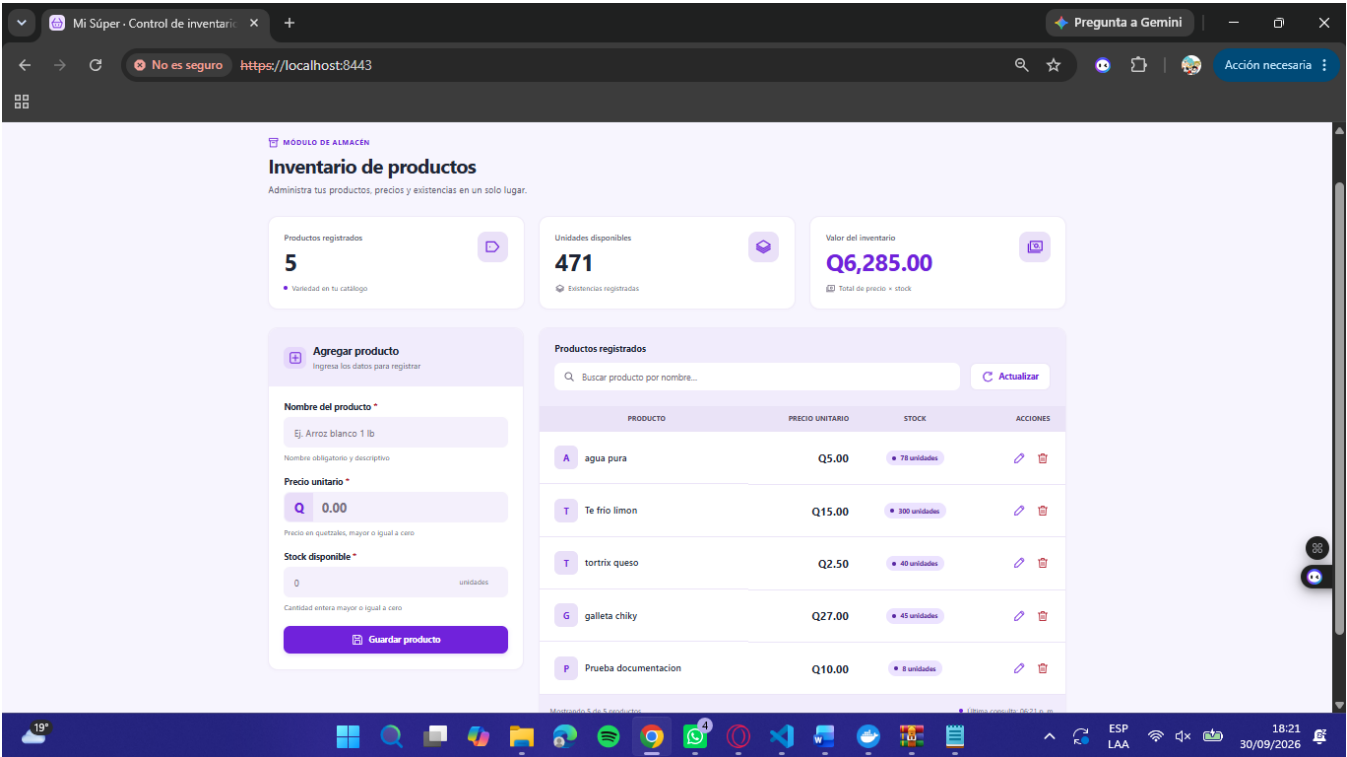
Funcionamiento: Muestra los servicios recreados y sus puertos.

Motivo: Confirmar disponibilidad antes de abrir Vue.

Resultado: Los cuatro contenedores aparecen Up y MongoDB y backend healthy.

Qué demuestra: La comprobación de persistencia se realiza sobre servicios nuevamente activos.

Datos conservados después de recrear



Evidencia 23. Datos conservados después de recrear.

Actividad: Consultar el producto sin volver a crearlo.

Comando o configuración: Recarga de <https://localhost:8443>

Funcionamiento: Vue obtiene los registros del backend, que usa la misma base almacenada en el volumen.

Motivo: Comparar el resultado con la captura previa de actualización.

Resultado: Prueba documentación conserva precio Q10.00 y stock 8; siguen 5 productos, 471 unidades y Q6,285.00.

Qué demuestra: Los datos sobreviven a la eliminación y recreación de los contenedores.

Etapa 15 HTTPS y redirección

La aplicación se accede por `https://localhost:8443`, como muestran las figuras 4 y 5. El puerto HTTP 8082 se utiliza para redirigir a HTTPS. Los puertos internos 443 y 80 del gateway se corresponden con esos puertos publicados del host.

```
curl -I http://localhost:8082
```

`curl -I` solicita los encabezados para observar la redirección sin descargar el cuerpo. La captura devuelve HTTP/1.1 301 Moved Permanently y Location: `https://localhost:8443/`, conforme a la configuración de Nginx.

```
curl --cacert nginx/certs/server.crt \  
-i https://localhost:8443/api/productos
```

`--cacert` utiliza el certificado local como referencia de confianza para esta conexión y `-i` muestra los encabezados junto con el cuerpo. La captura actual devuelve HTTP/1.1 200 OK y los cuatro productos en JSON. Coinciden con los registros consultados directamente mediante `mongosh` y con la interfaz previa al producto temporal.

Actividad: Consultar la API mediante TLS a través de Nginx.

Comando o configuración: `curl --cacert nginx/certs/server.crt -i https://localhost:8443/api/productos`

Funcionamiento: Establece HTTPS, verifica con el certificado indicado y consulta Flask por proxy.

Motivo: Comprobar certificado, enrutamiento y acceso a MongoDB en una sola petición.

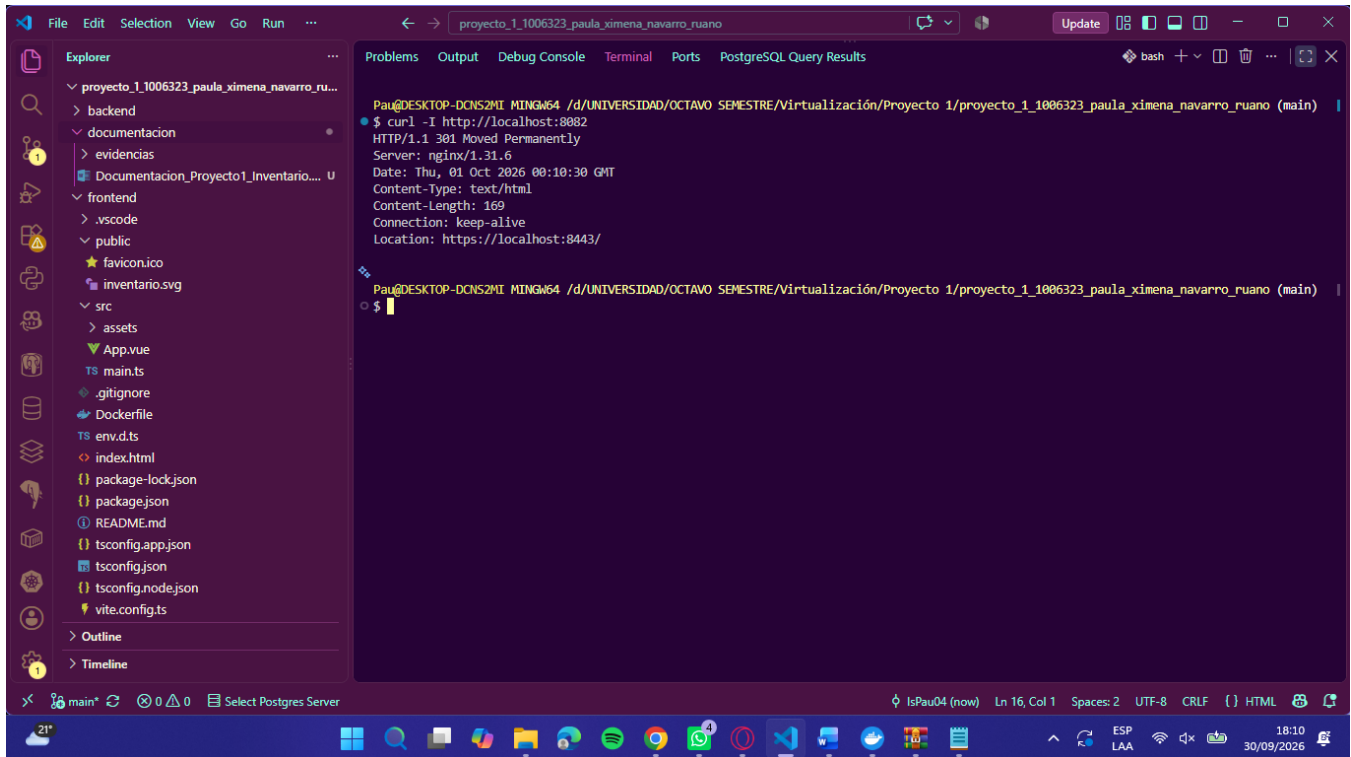
Resultado: 200 OK y JSON con agua pura, Te frio limon, tortrix queso y galleta chiky.

Qué demuestra: la integración responde por HTTPS y entrega los documentos almacenados en MongoDB a través de Nginx y Flask.

Advertencia del navegador

El navegador muestra No es seguro junto a la dirección HTTPS porque el certificado es autofirmado y no está avalado por una autoridad reconocida por el navegador. La conexión utiliza HTTPS; la inspección con OpenSSL de la etapa 10 documenta sujeto, SAN y vigencia. Para un servicio público corresponde utilizar un certificado emitido por una autoridad confiable.

Redirección de HTTP a HTTPS



The screenshot shows a Visual Studio Code window with a terminal open. The terminal is running a command to test an HTTP redirect. The output shows a 301 Moved Permanently status from Nginx, with the Location header pointing to the HTTPS version of the site.

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |  
$ curl -I http://localhost:8082  
HTTP/1.1 301 Moved Permanently  
Server: nginx/1.31.6  
Date: Thu, 01 Oct 2026 00:10:30 GMT  
Content-Type: text/html  
Content-Length: 169  
Connection: keep-alive  
Location: https://localhost:8443/  
  
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |  
$
```

Evidencia 24. Redirección de HTTP a HTTPS.

Actividad: Consultar la respuesta del puerto HTTP.

Comando o configuración: `curl -I http://localhost:8082`

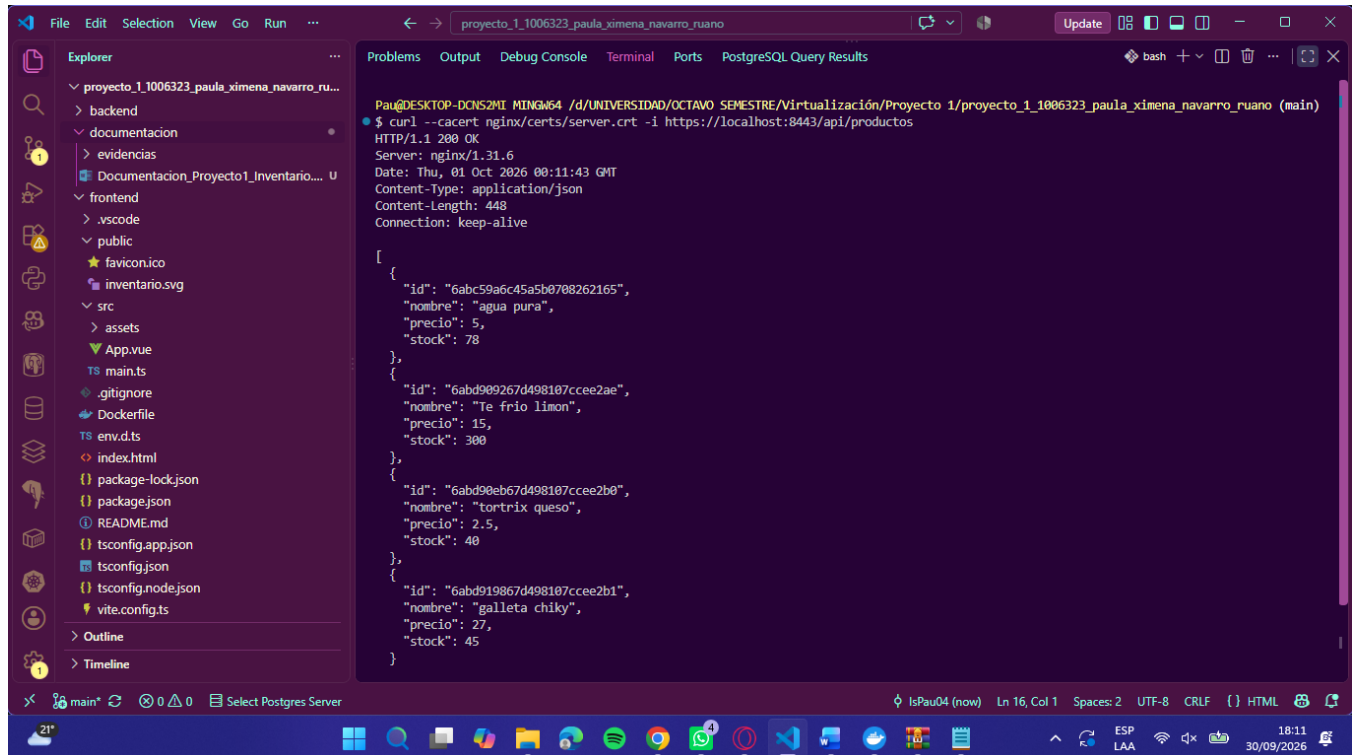
Funcionamiento: Solicita encabezados; Nginx devuelve la redirección configurada.

Motivo: Comprobar el cambio de esquema y puerto.

Resultado: 301 Moved Permanently y Location: `https://localhost:8443/`.

Qué demuestra: El gateway dirige el acceso HTTP hacia HTTPS.

Consulta de productos por HTTPS



The screenshot shows a VS Code editor with a terminal window open. The terminal displays the execution of a curl command to fetch product data via HTTPS. The response is a JSON array containing four product objects, each with an id, nombre, precio, and stock.

```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ curl --cacert nginx/certs/server.crt -i https://localhost:8443/api/productos
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Thu, 01 Oct 2026 00:11:43 GMT
Content-Type: application/json
Content-Length: 448
Connection: keep-alive

[
  {
    "id": "6abc59a6c45a5b0708262165",
    "nombre": "agua pura",
    "precio": 5,
    "stock": 78
  },
  {
    "id": "6abd909267d498107ccee2ae",
    "nombre": "Te frio limon",
    "precio": 15,
    "stock": 300
  },
  {
    "id": "6abd90eb67d498107ccee2b0",
    "nombre": "tortrix queso",
    "precio": 2.5,
    "stock": 40
  },
  {
    "id": "6abd919867d498107ccee2b1",
    "nombre": "galleta chiky",
    "precio": 27,
    "stock": 45
  }
]
```

Evidencia 25. Consulta de productos por HTTPS.

Actividad: Consultar la API con verificación del certificado.

Comando o configuración: `curl --cacert nginx/certs/server.crt -i https://localhost:8443/api/productos`

Funcionamiento: curl establece TLS con el certificado indicado; Nginx enruta la solicitud hacia Flask.

Motivo: Verificar HTTPS, proxy y acceso a datos en una petición.

Resultado: 200 OK y JSON con los cuatro productos registrados.

Qué demuestra: La API responde a través del gateway mediante HTTPS.

Etapla 16 Logs y diagnóstico

```
docker compose logs --tail=100
docker compose logs --tail=100 backend gateway
docker logs --tail=100 mongodb
docker compose exec gateway nginx -t
```

`docker compose logs` reúne los registros de los servicios del proyecto. `--tail=100` limita la salida reciente para hacerla legible. La segunda variante revisa backend y gateway, que participan en cada petición de la interfaz. `docker logs` consulta directamente un contenedor por nombre. `nginx -t` revisa la configuración, los certificados y la sintaxis dentro del gateway.

En Flask puedo observar solicitudes y excepciones de la API. En Nginx puedo revisar accesos, errores de upstream y fallos al cargar certificados. En MongoDB puedo revisar inicio del motor y problemas de almacenamiento o conexión. Un mensaje de acceso no demuestra por sí solo que toda la operación CRUD haya finalizado correctamente: debo contrastarlo con el estado HTTP y el resultado visible.

Relación entre síntomas y servicios

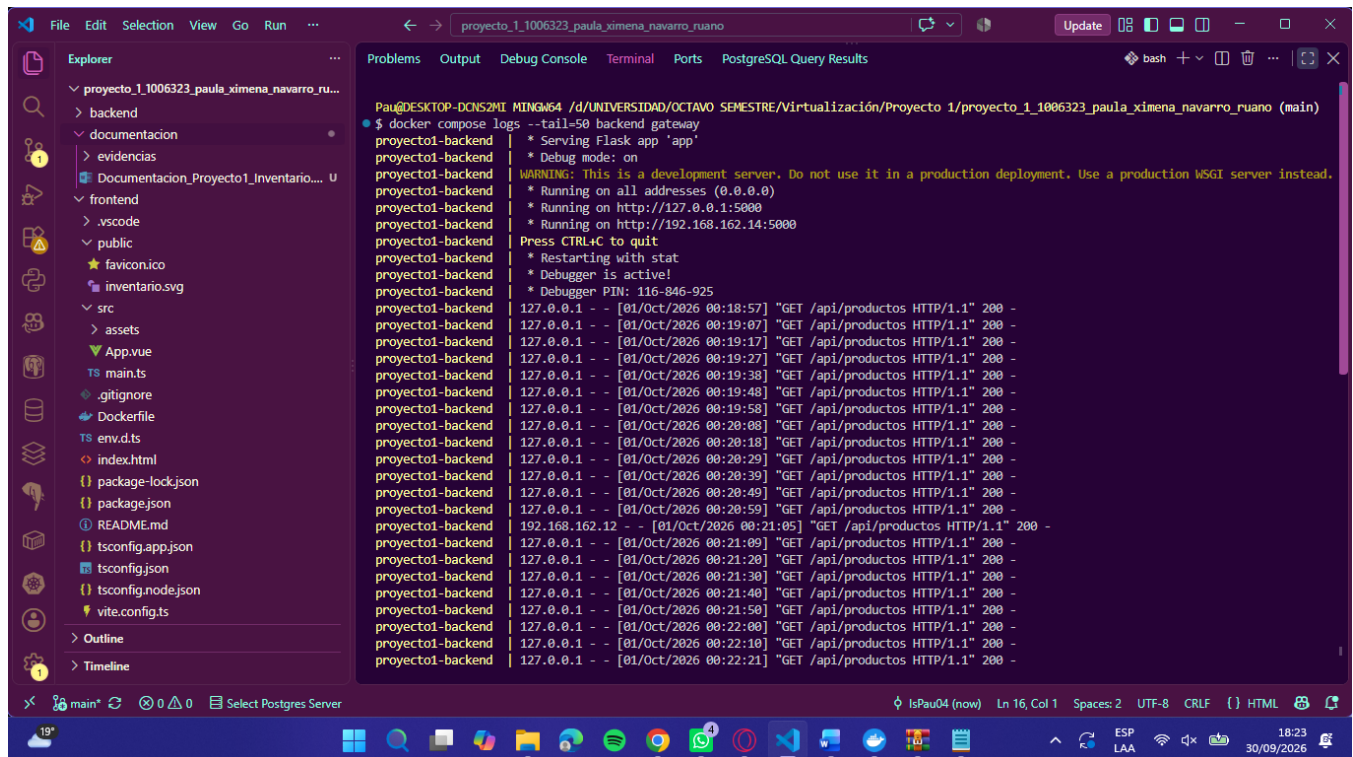
Síntoma	Revisión inicial
Página inaccesible	Estado de gateway, puertos y certificados
Interfaz carga pero no lista productos	Respuesta <code>/api/productos</code> , logs de gateway y backend
Backend unhealthy	Consulta Flask y disponibilidad de MongoDB
Producto desaparece después de recrear	Nombre del volumen y montaje <code>/data/db</code>
Nginx no inicia	<code>nginx -t</code> y rutas montadas

Las capturas finales de logs muestran el inicio del backend y del gateway, solicitudes GET con respuesta 200 y una solicitud DELETE con respuesta 200. Las solicitudes a 127.0.0.1 en Flask corresponden a consultas locales, como el healthcheck; las peticiones del gateway aparecen desde su IP interna. El aviso del servidor de desarrollo de Flask se conserva como característica del despliegue académico.

Comportamiento si un servicio falla

Si MongoDB deja de responder, Flask no podrá completar el acceso a datos. Si Flask falla, el proxy podrá devolver un error y Vue mostrará un aviso. Si el frontend falla, Nginx no podrá servir la interfaz. La política `unless-stopped` actúa ante la salida del proceso; un estado `unhealthy` por sí solo no obliga a Docker a reiniciar el contenedor. `depends_on` organiza el arranque inicial y no constituye supervisión continua de toda la cadena.

Registros del backend Flask



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$ docker compose logs --tail=50 backend gateway
projecto1-backend | * Serving Flask app 'app'
projecto1-backend | * Debug mode: on
projecto1-backend | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
projecto1-backend | * Running on all addresses (0.0.0.0)
projecto1-backend | * Running on http://127.0.0.1:5000
projecto1-backend | * Running on http://192.168.162.14:5000
projecto1-backend | Press CTRL+C to quit
projecto1-backend | * Restarting with stat
projecto1-backend | * Debugger is active!
projecto1-backend | * Debugger PIN: 116-846-925
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:18:57] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:19:07] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:19:17] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:19:27] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:19:38] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:19:48] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:19:58] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:20:08] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:20:18] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:20:29] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:20:39] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:20:49] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:20:59] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 192.168.162.12 - - [01/Oct/2026 00:21:05] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:21:09] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:21:20] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:21:30] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:21:40] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:21:50] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:22:00] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:22:10] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:22:21] "GET /api/productos HTTP/1.1" 200 -
```

Evidencia 26. Registros del backend Flask.

Actividad: Consultar los registros recientes del backend.

Comando o configuración: docker compose logs --tail=50 backend gateway

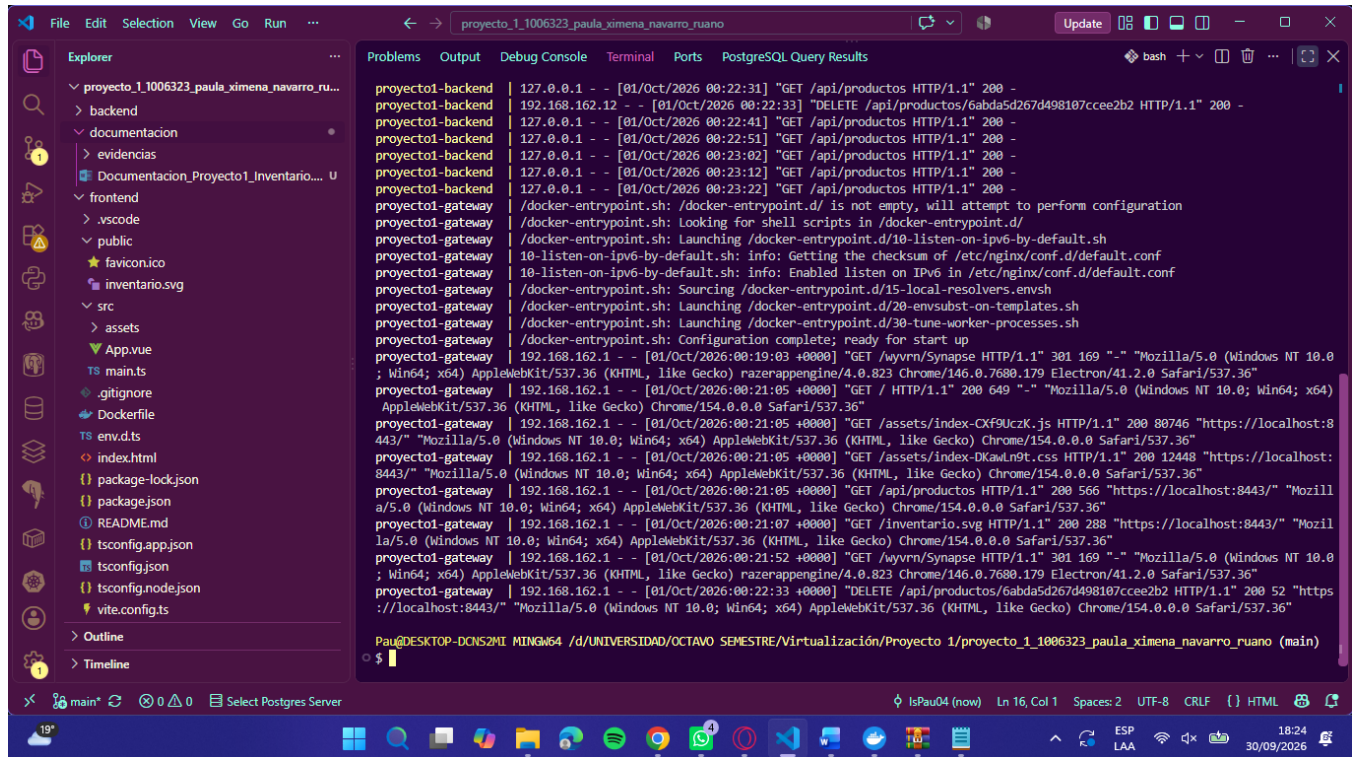
Funcionamiento: logs reúne la salida de los servicios; --tail limita las líneas por servicio.

Motivo: Identificar arranque, solicitudes y códigos HTTP.

Resultado: Se ven el inicio de Flask y solicitudes GET /api/productos con respuesta 200.

Qué demuestra: El backend atiende consultas; la captura histórica también conserva el aviso del servidor de desarrollo.

Registros del gateway y eliminación



```
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:22:31] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 192.168.162.12 - - [01/Oct/2026 00:22:33] "DELETE /api/productos/6abda5d267d498107ccee2b2 HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:22:41] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:22:51] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:23:02] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:23:12] "GET /api/productos HTTP/1.1" 200 -
projecto1-backend | 127.0.0.1 - - [01/Oct/2026 00:23:22] "GET /api/productos HTTP/1.1" 200 -
projecto1-gateway | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
projecto1-gateway | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
projecto1-gateway | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
projecto1-gateway | 10-listen-on-ipv6-by-default.sh: Info: Getting the checksum of /etc/nginx/conf.d/default.conf
projecto1-gateway | 10-listen-on-ipv6-by-default.sh: Info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
projecto1-gateway | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
projecto1-gateway | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
projecto1-gateway | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
projecto1-gateway | /docker-entrypoint.sh: Configuration complete; ready for start up
projecto1-gateway | 192.168.162.1 - - [01/Oct/2026:00:19:03 +0000] "GET /wvrrn/Synapse HTTP/1.1" 301 169 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) razerappengine/4.0.823 Chrome/146.0.7680.179 Electron/41.2.0 Safari/537.36"
projecto1-gateway | 192.168.162.1 - - [01/Oct/2026:00:21:05 +0000] "GET / HTTP/1.1" 200 649 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36"
projecto1-gateway | 192.168.162.1 - - [01/Oct/2026:00:21:05 +0000] "GET /assets/index-CXf9UczK.js HTTP/1.1" 200 80746 "https://localhost:8443/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36"
projecto1-gateway | 192.168.162.1 - - [01/Oct/2026:00:21:05 +0000] "GET /assets/index-DKawLn9t.css HTTP/1.1" 200 12448 "https://localhost:8443/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36"
projecto1-gateway | 192.168.162.1 - - [01/Oct/2026:00:21:05 +0000] "GET /api/productos HTTP/1.1" 200 566 "https://localhost:8443/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36"
projecto1-gateway | 192.168.162.1 - - [01/Oct/2026:00:21:07 +0000] "GET /inventario.svg HTTP/1.1" 200 288 "https://localhost:8443/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36"
projecto1-gateway | 192.168.162.1 - - [01/Oct/2026:00:21:52 +0000] "GET /wvrrn/Synapse HTTP/1.1" 301 160 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) razerappengine/4.0.823 Chrome/146.0.7680.179 Electron/41.2.0 Safari/537.36"
projecto1-gateway | 192.168.162.1 - - [01/Oct/2026:00:22:33 +0000] "DELETE /api/productos/6abda5d267d498107ccee2b2 HTTP/1.1" 200 52 "https://localhost:8443/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36"

Pau@DESKTOP-DCNS2MI MINGW64 /d:/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main)
$
```

Evidencia 27. Registros del gateway y eliminación.

Actividad: Revisar la continuación de los registros.

Comando o configuración: docker compose logs --tail=50 backend gateway

Funcionamiento: Muestra accesos de Nginx y peticiones atendidas por Flask.

Motivo: Relacionar la actividad web con el resultado CRUD.

Resultado: Aparecen GET y DELETE con respuesta 200, junto con el inicio del gateway.

Qué demuestra: Los logs permiten identificar rutas, métodos, clientes y respuestas durante las pruebas.

Publicación del código y entrega

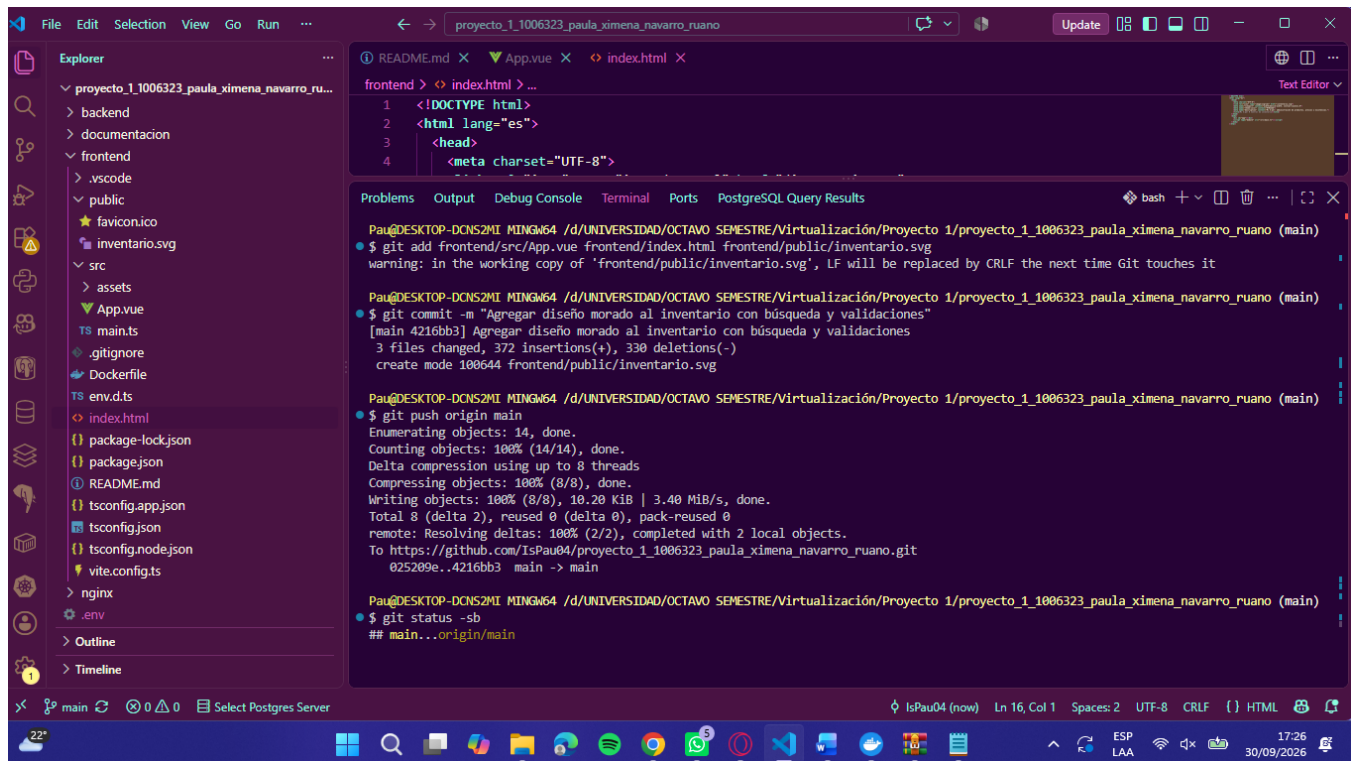


Figura 8. Commit del diseño final, push a main y estado sincronizado.

Actividad: Guardar y publicar las mejoras del frontend.

Comando o configuración: git add de los tres archivos del frontend; git commit; git push origin main; git status -sb.

Funcionamiento: add prepara cambios, commit registra una versión, push la publica y status compara la rama con origin.

Motivo: Conservar el trabajo y permitir reconstruir el código después del apagado del equipo.

Resultado: Commit 4216bb3 publicado; status muestra main...origin/main sin cambios pendientes en la captura.

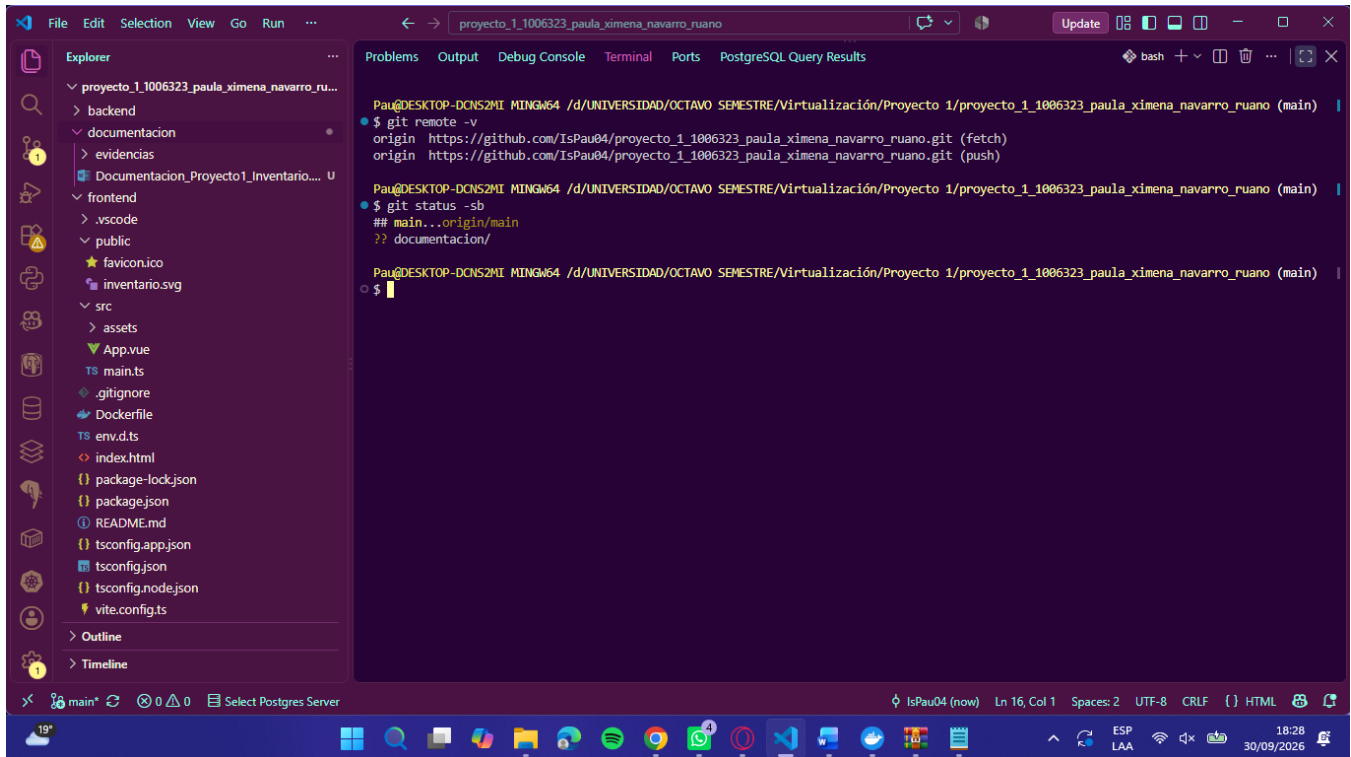
Qué demuestra: La versión del frontend visible en ese momento estaba guardada y sincronizada con GitHub.

Contenido de la entrega

La carpeta final debe contener código, Dockerfiles, Compose, .env local, Nginx, certificado y llave local, README, este informe, diagrama y evidencias. El repositorio público incluye .env.example y los archivos necesarios para regenerar el certificado; .env y server.key permanecen excluidos. Para la entrega local al docente se revisa que esos archivos existan, sin publicarlos en GitHub.

El README actualizado incluye tecnologías, requisitos, diagrama Mermaid, directorios, servicios, IP, red, MongoDB, volumen, variables, plantilla de Nginx, certificados, instalación, construcción, ejecución, detención, reconstrucción, pruebas, persistencia y una tabla de problemas con sus soluciones. La documentación y las capturas complementan esa guía dentro de documentacion.

Repositorio remoto y estado de Git



```
Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$ git remote -v
origin https://github.com/IsPau04/proyecto_1_1006323_paula_ximena_navarro_ruano.git (fetch)
origin https://github.com/IsPau04/proyecto_1_1006323_paula_ximena_navarro_ruano.git (push)

Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$ git status -sb
## main...origin/main
?? documentacion/

Pau@DESKTOP-DCNS2MI MINGW64 /d/UNIVERSIDAD/OCTAVO SEMESTRE/Virtualización/Proyecto 1/proyecto_1_1006323_paula_ximena_navarro_ruano (main) |
$
```

Evidencia 28. Repositorio remoto y estado de Git.

Actividad: Comprobar el destino de publicación y los cambios locales.

Comando o configuración: git remote -v y git status -sb

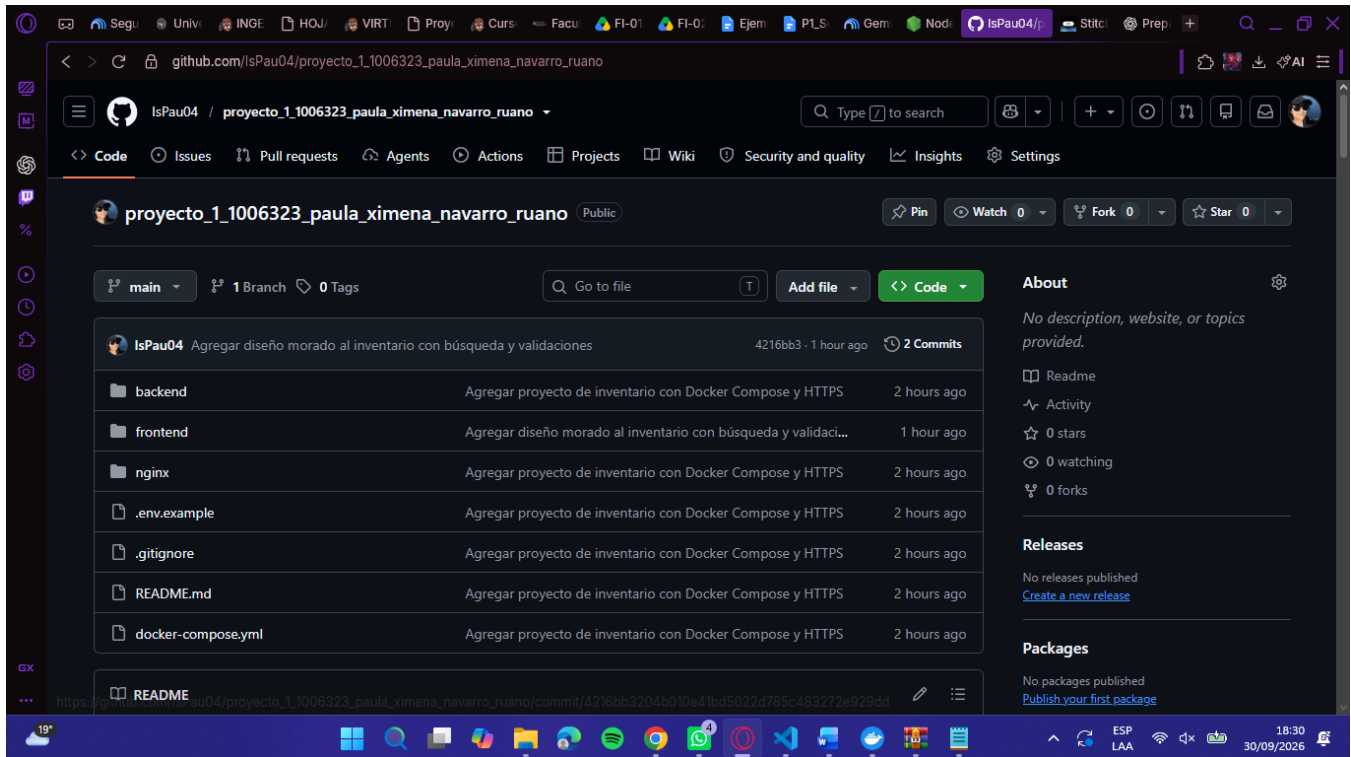
Funcionamiento: remote muestra las URL; status compara main con origin/main y lista archivos nuevos.

Motivo: Verificar dónde se respalda el código.

Resultado: origin corresponde al repositorio de IsPau04; main sigue a origin/main; documentacion aparece sin seguimiento.

Qué demuestra: El código tiene un remoto configurado; la documentación local requiere un commit posterior.

Código publicado en GitHub



Evidencia 29. Código publicado en GitHub.

Actividad: Abrir el repositorio publicado.

Comando o configuración: Repositorio IsPau04/proyecto_1_1006323_paula_ximena_navarro_ruano

Funcionamiento: GitHub muestra la rama main, los archivos y los commits.

Motivo: Comprobar la disponibilidad del código fuente.

Resultado: Se ven backend, frontend, nginx, .env.example, .gitignore, README y docker-compose.yml.

Qué demuestra: La versión del código está publicada; esta captura no acredita la subida posterior de la documentación.

Problemas resueltos y conclusiones

Ajustes realizados durante la implementación

El puerto 8080 del equipo no estaba disponible para la publicación prevista. Cambié el acceso HTTP a 8082 y mantuve HTTPS en 8443. El archivo de Nginx redirige hacia el puerto publicado correcto. Los servicios de datos y API conservan sus puertos internos sin publicarlos al host.

OpenSSL intentaba usar una configuración predeterminada que no se encontraba en la ruta del equipo. Incorporé openssl.cnf al proyecto y utilicé -config para seleccionarlo explícitamente. También añadí los nombres alternativos localhost y 127.0.0.1.

La primera instancia del gateway montaba archivos de la antigua carpeta del repositorio del curso. Al migrar a la carpeta final, reconstruí y recreé los servicios con Compose. La inspección posterior confirmó los montajes de nginx.conf y certs desde la ruta correcta.

La llave privada inicialmente no coincidía con una regla de exclusión. Añadí nginx/certs/*.key a .gitignore y git check-ignore confirmó que .env y server.key están ignorados. La lista preparada para commit incluía el certificado público y openssl.cnf, sin la llave privada.

Lo aprendido

Este proyecto me permitió separar la interfaz, la lógica y los datos sin perder la comunicación entre ellos. Compose concentra la definición de los servicios; la red personalizada y las IP fijas organizan su conexión. Nginx proporciona una entrada web común y aplica HTTPS. El volumen separa los productos del ciclo de vida de MongoDB.

Las pruebas documentadas muestran la solución operando en contenedores, el CRUD desde Vue, la conservación de productos al recrear los servicios y el acceso por HTTPS. La red, el volumen, las variables, las imágenes y el proxy se verificaron con comandos de inspección. El código mantiene un alcance académico local; la validación del servidor y la ejecución con debug pueden fortalecerse para un despliegue público.

Material consultado para este informe

Enunciado del Proyecto 1, Infraestructura Containerizada con Docker, curso de Virtualización, fecha de entrega 1 de octubre de 2026. Archivo proporcionado: proyecto 1.pdf. Guía de configuración del proyecto proporcionada: configuracion proyecto 1.pdf. Código final local, README y archivos de configuración del repositorio identificado. Capturas originales y salidas de terminal compartidas durante la implementación.

Resumen de las evidencias de la solución

Las capturas incorporadas se distribuyen en las etapas del informe y respaldan las verificaciones de configuración y funcionamiento. La tabla siguiente permite localizar el contenido documentado para la revisión.

Etapas	Evidencia incluida
2	Ruta de trabajo y estructura mediante pwd y ls -la
3 y 11	Inicio y fin del build; imágenes propias en docker images
5	Documentos en mongosh; volumen y montaje en /data/db
6	Variables iniciales y finales de MongoDB, Flask, puertos, IP y recursos externos; validación de Compose.
7	Subred, gateway y direcciones de los cuatro contenedores
9	nginx -t sobre el gateway final
10	Inspección inicial, generación del nuevo par y verificación del certificado regenerado con HTTPS.
12	Estado y puertos de los cuatro contenedores
13	Consulta, creación, modificación y eliminación desde Vue
14	Datos previos, down, up, ps y datos conservados
15	Acceso HTTPS, API con certificado y redirección 301
16	Logs de Flask y del gateway con interpretación

Comandos para la revisión práctica

```
docker compose ps
docker images
docker network ls
docker network inspect proyecto_network
docker volume ls
docker compose exec gateway nginx -t
docker compose logs --tail=50 backend gateway
openssl x509 -in nginx/certs/server.crt \
  -noout -subject -dates -ext subjectAltName
curl -I http://localhost:8082
```

Estos comandos permiten revisar el estado de la infraestructura sin eliminar datos. La prueba de persistencia se realiza aparte con la secuencia documentada en la etapa 14. El informe y las evidencias se guardan dentro de documentacion para acompañar el código del proyecto.